

Protocolli di comunicazione seriale

UART/USART · SPI · I2C

*Guida di studio per colloquio tecnico embedded
Con tabella comparativa, esempi STM32 HAL / ESP-IDF, cenni a RS-485,
Modbus RTU e CAN, domande tipiche con risposte*

Indice

1. Concetti comuni della comunicazione seriale
2. UART / USART
3. SPI
4. I2C
5. Tabella comparativa e criteri di scelta
6. Cenni a RS-485, Modbus RTU e CAN
7. Domande tipiche da colloquio (con risposte)
8. Riferimenti
9. Glossario degli acronimi

1. Concetti comuni della comunicazione seriale

Nella comunicazione **seriale** i bit vengono trasmessi uno alla volta su una singola linea (o coppia di linee), al contrario della comunicazione parallela che usa un filo per ogni bit. Il vantaggio principale è la riduzione del numero di pin e di piste: per questo tutte le periferiche di un microcontrollore moderno (sensori, memorie, display, radio) parlano seriale. I tre protocolli trattati coprono la stragrande maggioranza delle interconnessioni a livello di scheda.

1.1 Sincrono vs asincrono

- **Sincrono** (SPI, I2C, USART in modo sincrono): esiste una linea di **clock** condivisa che scandisce il campionamento dei bit. Il ricevitore non deve conoscere in anticipo la velocità: legge il dato sul fronte del clock. Costa un pin in più ma permette velocità elevate e nessun problema di deriva degli oscillatori.
- **Asincrono** (UART): nessun clock condiviso. Trasmettitore e ricevitore devono essere configurati sullo stesso **baud rate** e si sincronizzano sul **bit di start** di ogni frame. Richiede oscillatori sufficientemente precisi e bit di start/stop che riducono l'efficienza (10 bit trasmessi per 8 bit utili nel formato 8N1).

Acronimi:

- **SPI** = Serial Peripheral Interface, interfaccia seriale per periferiche
- **I2C** = Inter-Integrated Circuit, bus tra circuiti integrati
- **USART** = Universal Synchronous/Asynchronous Receiver/Transmitter, versione con modo sincrono opzionale
- **UART** = Universal Asynchronous Receiver/Transmitter, ricetrasmettitore asincrono universale

1.2 Duplex

- **Full duplex**: trasmissione e ricezione simultanee su linee separate (UART con TX/RX, SPI con MOSI/MISO).
- **Half duplex**: una sola linea dati condivisa, si alterna la direzione (I2C, RS-485 a 2 fili, SPI a 3 fili, UART single-wire).

Acronimi:

- **TX** = Transmit, linea di trasmissione
- **RX** = Receive, linea di ricezione
- **MOSI** = Master Out Slave In
- **MISO** = Master In Slave Out
- **RS-485** = Recommended Standard 485, standard elettrico differenziale multidrop

1.3 Topologie e indirizzamento

- **Punto-punto**: due soli nodi (UART). Nessun indirizzo necessario.
- **Bus con selezione hardware**: un master seleziona lo slave con una linea dedicata Chip Select (SPI). Un pin per ogni slave.
- **Bus con indirizzamento software**: tutti i nodi condividono le stesse linee e ognuno risponde solo al proprio indirizzo (I2C, Modbus, CAN).

Acronimi:

- **CAN** = Controller Area Network

1.4 Livelli elettrici

A livello di scheda i segnali sono **single-ended** a livello logico del microcontrollore (tipicamente 3,3 V o 5 V, detti impropriamente 'TTL'). Non sono adatti a lunghe distanze o ambienti rumorosi: per uscire dalla scheda si usano **transceiver** che convertono in standard più robusti (RS-232 con tensioni bipolari fino a +/-15 V; RS-485 e CAN **differenziali**, dove l'informazione è la differenza tra due fili, immune ai disturbi di modo comune). Un aspetto spesso chiesto al colloquio: **UART è**

un protocollo/periferica, RS-232 e RS-485 sono standard elettrici che trasportano tipicamente frame UART.

Acronimi:

- **TTL** = Transistor-Transistor Logic, livelli logici 0/5 V (per estensione 0/3,3 V)
- **RS-232** = Recommended Standard 232, standard elettrico EIA/TIA punto-punto

1.5 Terminologia master/slave

Recentemente la terminologia sta migrando verso **controller/target** (I2C, dal 2021 nella specifica NXP) e **controller/peripheral** o **main/sub** (SPI). In questo documento si usa master/slave perché è ancora quella prevalente nei datasheet e nei reference manual; è utile conoscere entrambe.

Acronimi:

- **NXP** = NXP Semiconductors, ex divisione semiconduttori Philips

Al colloquio: la domanda di apertura classica

"Qual è la differenza tra UART, SPI e I2C?" - Risposta in tre righe: **UART** è asincrono, punto-punto, 2 fili (TX/RX), full duplex, richiede baud rate concordato. **SPI** è sincrono, master-slave con Chip Select hardware, 4 fili, full duplex, molto veloce (decine di MHz), senza indirizzi né acknowledge. **I2C** è sincrono, multi-slave (e multi-master) su 2 fili open-drain con pull-up, half duplex, indirizzamento a 7 bit e acknowledge per ogni byte, velocità tipica 100-400 kHz. Poi si approfondisce secondo la direzione che prende l'intervistatore.

2. UART / USART

UART = Universal Asynchronous Receiver/Transmitter. È la periferica hardware che serializza e deserializza i byte secondo un formato asincrono. **USART** aggiunge una modalità sincrona (con clock in uscita) e spesso altre estensioni. Non esiste un 'ente' che definisce il protocollo UART: è una convenzione de facto, mentre gli standard formali riguardano i livelli elettrici (RS-232, RS-485, RS-422).

Acronimi:

- **RS-422** = Recommended Standard 422, standard elettrico differenziale punto-punto

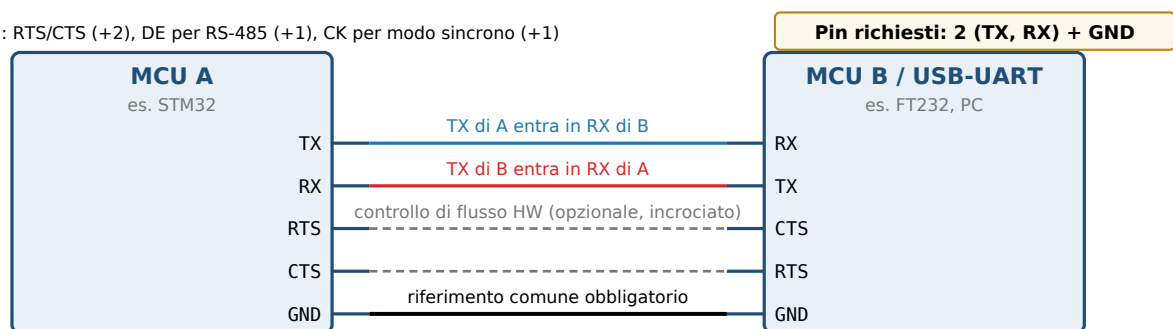
2.1 Linee

- **TX** (trasmissione) e **RX** (ricezione): collegate a incrocio (TX di A -> RX di B e viceversa). Errore classico da principiante: collegare TX con TX.
- **GND** (Ground, massa) comune, indispensabile.
- Opzionali: **RTS/CTS** per il controllo di flusso hardware; **DE** per pilotare il driver enable di un transceiver RS-485; **CK** (clock) solo in USART sincrona.

Acronimi:

- **RTS** = Request To Send, richiesta di trasmettere
- **CTS** = Clear To Send, via libera alla trasmissione

Opzionali: RTS/CTS (+2), DE per RS-485 (+1), CK per modo sincrono (+1)



UART: collegamento punto-punto. Le linee sono incrociate: l'uscita di uno è l'ingresso dell'altro.

Figura 1 - Collegamento UART tra due dispositivi: 2 pin dati + GND; RTS/CTS opzionali.

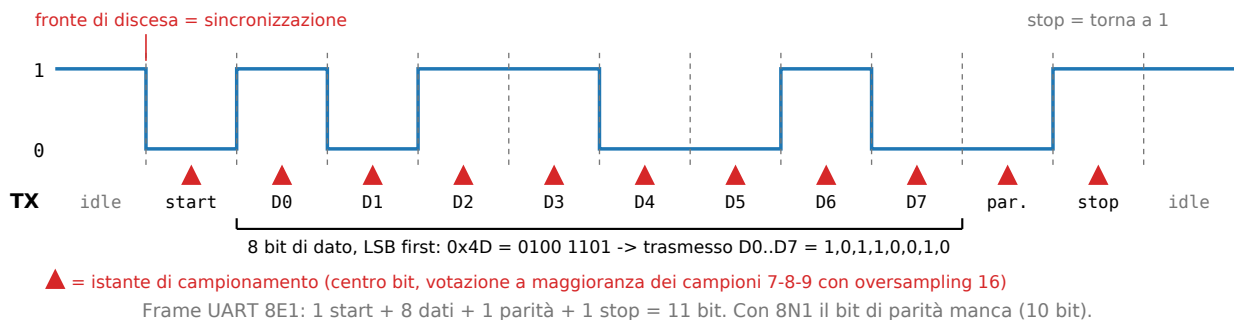
2.2 Struttura del frame

La linea a riposo (idle) è a livello **alto**. Un frame è composto da:

Campo	Durata	Descrizione
Start bit	1 bit	Transizione alto -> basso. Il ricevitore la rileva e da lì conta i tempi di bit. È il meccanismo di sincronizzazione.
Data bits	5-9 bit	Tipicamente 8. Trasmessi LSB first (a differenza di SPI/I2C che sono MSB first). Il 9° bit è usato per la parità o per indirizzamento multiprocessore.
Parity bit	0-1 bit	Opzionale: even, odd (talvolta mark/space). Rileva un numero impari di bit errati. Modbus RTU usa even di default.
Stop bit	1, 1.5 o 2 bit	Livello alto. Garantisce un tempo minimo di idle prima del prossimo start e permette al ricevitore di verificare il framing.

Acronimi:

- **LSB** = Least Significant Bit, bit meno significativo
- **MSB** = Most Significant Bit, bit più significativo
- **RTU** = Remote Terminal Unit, variante binaria di Modbus



Frame UART 8E1: 1 start + 8 dati + 1 parità + 1 stop = 11 bit. Con 8N1 il bit di parità manca (10 bit).

Figura 2 - Frame UART con parità pari: il ricevitore si sincronizza sullo start bit e campiona al centro di ogni bit.

La notazione compatta è **8N1** (8 dati, No parity, 1 stop) o **8E1** (Even parity). Con 8N1 l'efficienza è $8/10 = 80\%$: a 115200 baud si trasferiscono al massimo 11520 byte/s.

Calcolo: da baud rate a byte/s (formato 8N1)

Bit per frame = start + dati + parità + stop = $1 + 8 + 0 + 1 = 10$ bit

Efficienza = bit utili / bit trasmessi = $8 / 10 = 80\%$

Tempo di un bit: $T_{\text{bit}} = 1 / \text{baud} = 1 / 115200 \text{ s} = 8,68 \mu\text{s}$

Tempo di un frame: $T_{\text{frame}} = 10 \times T_{\text{bit}} = 10 / 115200 \text{ s} = 86,8 \mu\text{s}$

Throughput massimo = baud / bit per frame = $115200 / 10 = 11520 \text{ byte/s}$ ($= 1 / T_{\text{frame}}$)

Con 8E1 (11 bit per frame): $115200 / 11 = 10473 \text{ byte/s}$, efficienza $8/11 = 72,7\%$

Con 8N2 o 8E2 (12 bit): $115200 / 12 = 9600 \text{ byte/s}$, efficienza $66,7\%$

Il valore è un massimo teorico: ogni gap tra un frame e il successivo lo riduce ulteriormente.

2.3 Baud rate, oversampling e tolleranza

Il **baud rate** è il numero di simboli al secondo; nel caso UART coincide con i bit/s perché ogni simbolo è un bit. Valori standard: 9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600. Il ricevitore campiona ogni bit con un clock interno a frequenza multipla (**oversampling** x16 tipico, x8 per raggiungere baud rate elevati) e decide il valore leggendo al centro del bit (spesso votazione a maggioranza sui campioni 7, 8, 9). Il baud rate effettivo dipende dal clock della periferica e dal divisore: $\text{baud} = f_{\text{clk}} / (\text{oversampling} \times \text{BRR})$. Non sempre il divisore è intero, quindi esiste un **errore di baud rate**; l'errore complessivo tra i due nodi deve restare sotto circa il 2-3 % (il valore esatto dipende da oversampling e lunghezza del frame; il reference manual STM32 riporta tabelle di tolleranza per ciascun caso). Per questo con oscillatori RC interni imprecisi si preferiscono baud rate bassi o si usa un quarzo.

Acronimi:

- **BRR** = Baud Rate Register, registro del divisore di baud rate (STM32)
- **RC** = Resistenza-Condensatore

Calcolo: baud rate effettivo ed errore di divisione (esempio STM32, $f_{\text{clk}} = 16 \text{ MHz}$, oversampling 16)

Divisore ideale: $\text{USARTDIV} = f_{\text{clk}} / (16 \times \text{baud}) = 16\,000\,000 / (16 \times 115200) = 8,6806$

Il registro BRR memorizza il divisore con 4 bit frazionari (passo 1/16): 8,6806 -> mantissa 8, frazione

$\text{round}(0,6806 \times 16) = \text{round}(10,9) = 11$ -> $\text{USARTDIV}_{\text{reale}} = 8 + 11/16 = 8,6875$

Baud effettivo = $f_{\text{clk}} / (16 \times \text{USARTDIV}_{\text{reale}}) = 16\,000\,000 / (16 \times 8,6875) = 115\,108 \text{ baud}$

Errore = $(\text{baud}_{\text{eff}} - \text{baud}_{\text{nom}}) / \text{baud}_{\text{nom}} = (115\,108 - 115\,200) / 115\,200 = -0,08\%$

Stesso calcolo a 921600 baud: $\text{USARTDIV} = 1,0851$ -> $1 + 1/16 = 1,0625$ -> 941 176 baud -> errore **+2,1 %**: già al limite. Da qui l'uso di oversampling 8 o di un clock diverso.

Il budget di errore va diviso tra i due nodi: se il PC ha lo 0,2 % e la tolleranza totale è 2 %, all'MCU (compreso il suo quarzo, tipicamente +/-50 ppm = 0,005 %) restano 1,8 %.

Acronimi:

- **PC** = Personal Computer
- **MCU** = Microcontroller Unit, microcontrollore
- **ppm** = parti per milione

Calcolo: perché la tolleranza è dell'ordine di qualche percento

Il ricevitore si sincronizza sullo start bit e campiona al centro di ogni bit. L'errore di frequenza si accumula bit dopo bit.

Ultimo bit da campionare in $8N1$ = stop bit, il 10°: il suo centro cade dopo $9,5 T_{bit}$ dal fronte dello start.

Il campione resta dentro il bit corretto finché la deriva è $< 0,5 T_{bit}$: $\text{errore}_{max} = 0,5 / 9,5 = 5,3 \%$ (limite teorico ideale).

Con 9 bit dati + parità (11 bit per frame): $0,5 / 10,5 = 4,8 \%$.

Sottraendo l'incertezza del campionamento (con oversampling 16 il centro è noto a $\pm 1/16$ di bit, $\sim 6 \%$ di un bit), il jitter e la somma degli errori di entrambi i nodi si ottiene il valore pratico di **2-3 %** complessivi (i reference manual STM32 riportano tabelle di tolleranza del solo ricevitore dell'ordine del 3-4 % per frame $8N1$ con oversampling 16, da cui va sottratto l'errore del trasmettitore).

2.4 Errori rilevabili

Flag	Causa	Cosa fare
Framing error (FE)	Stop bit letto a 0. Baud rate errato, rumore, break condition.	Scartare il byte, verificare configurazione/tolleranza.
Overrun error (ORE)	Nuovo byte arrivato prima che il firmware leggesse il precedente dal registro dati.	Usare interrupt/DMA/FIFO, ridurre latenza ISR. Su STM32 l'ORE blocca la ricezione fino a clear.
Parity error (PE)	Parità non corrispondente.	Scartare il frame; a livello applicativo richiedere ritrasmissione (es. CRC Modbus).
Noise error (NE)	I campioni di uno stesso bit non concordano.	Problema elettrico: layout, lunghezza cavo, ground.
Break / Idle	Linea bassa per più di un frame / linea alta per un frame intero.	Usati come segnalazione: l'IDLE è ottimo per rilevare la fine di un pacchetto a lunghezza variabile.

Acronimi:

- **DMA** = Direct Memory Access, trasferimento dati senza intervento della CPU
- **FIFO** = First In First Out, coda hardware
- **ISR** = Interrupt Service Routine, routine di servizio dell'interrupt
- **CRC** = Cyclic Redundancy Check, controllo di ridondanza ciclico

2.5 Controllo di flusso

- **Hardware (RTS/CTS)**: il ricevitore abbassa RTS quando il buffer è pieno; il trasmettitore sospende finché CTS torna attivo. Gestito dalla periferica in modo trasparente.
- **Software (XON/XOFF)**: caratteri di controllo $0x11/0x13$ nel flusso. Non adatto a dati binari.
- **Nessuno**: caso più comune negli embedded, affidandosi a buffer sufficienti e ritmi di trasmissione noti.

Acronimi:

- **XON/XOFF** = Transmit On / Transmit Off, caratteri di controllo di flusso software

2.6 Differenza UART vs USART

La USART può generare un clock sul pin CK e trasmettere in modo sincrono (compatibile con SPI in alcune implementazioni), eliminando il problema della tolleranza di baud rate. Le USART dei microcontrollori moderni integrano inoltre modalità specializzate: **LIN** (automotive), **IrDA**,

Smartcard (ISO 7816), **Modbus/RS-485** con driver enable automatico e timeout, **single-wire half duplex**, rilevamento automatico del baud rate, wake-up da low power. In pratica quasi tutte le periferiche 'UART' degli STM32 sono USART e vengono usate in modo asincrono.

Acronimi:

- **LIN** = Local Interconnect Network, bus seriale automotive
- **IrDA** = Infrared Data Association, comunicazione a infrarossi
- **ISO** = International Organization for Standardization

2.7 Quando si usa

- **Console di debug / log**: la prima cosa che si attiva su qualsiasi scheda (printf su UART).
- **Bootloader e programmazione**: il bootloader di sistema STM32 e quello ESP32 usano UART.
- **Moduli esterni**: GPS (NMEA), modem GSM/LTE (comandi AT), Bluetooth/Wi-Fi seriali, lettori RFID, stampanti.
- **Comunicazione tra due MCU** o MCU-PC (tramite convertitore USB-UART come FTDI, CP2102, CH340).
- **Bus di campo** tramite transceiver: RS-485 per Modbus RTU (energy meter, PLC, inverter), RS-232 per apparati legacy.

Acronimi:

- **GPS** = Global Positioning System
- **NMEA** = National Marine Electronics Association, formato testuale delle frasi GPS
- **GSM** = Global System for Mobile Communications
- **LTE** = Long Term Evolution
- **AT** = ATtention, prefisso dei comandi Hayes per modem
- **RFID** = Radio-Frequency IDentification
- **USB** = Universal Serial Bus
- **FTDI** = Future Technology Devices International, produttore di convertitori USB-seriale
- **PLC** = Programmable Logic Controller, controllore logico programmabile

Limiti: solo punto-punto (senza transceiver), velocità limitata (pochi Mbit/s), efficienza 80%, nessuna gestione di framing di pacchetto: la delimitazione dei messaggi va gestita a livello applicativo (terminatore, lunghezza in header, timeout/IDLE, COBS/SLIP).

Acronimi:

- **COBS** = Consistent Overhead Byte Stuffing, schema di framing per dati binari
- **SLIP** = Serial Line Internet Protocol, schema di framing

2.8 Esempi di codice

STM32 HAL - UART

```
// Configurazione tipica (generata da CubeMX): 115200 8N1, oversampling 16
huart1.Instance = USART1;
huart1.Init.BaudRate = 115200;
huart1.Init.WordLength = UART_WORDLENGTH_8B; // 9B se si usa la parità con 8 dati
huart1.Init.StopBits = UART_STOPBITS_1;
huart1.Init.Parity = UART_PARITY_NONE;
huart1.Init.Mode = UART_MODE_TX_RX;
huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart1.Init.OverSampling = UART_OVERSAMPLING_16;
HAL_UART_Init(&huart1);

// Trasmissione bloccante
HAL_UART_Transmit(&huart1, (uint8_t *) "OK\r\n", 4, 100);

// Ricezione a lunghezza variabile: DMA circolare + evento IDLE (pattern consigliato)
static uint8_t rx_buf[256];
HAL_UARTEx_ReceiveToIdle_DMA(&huart1, rx_buf, sizeof(rx_buf));
__HAL_DMA_DISABLE_IT(huart1.hdmarx, DMA_IT_HT); // evita callback a metà buffer

void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    // Size = byte ricevuti fino all'IDLE: qui si fa il parsing del frame
    process_frame(rx_buf, Size);
    HAL_UARTEx_ReceiveToIdle_DMA(huart, rx_buf, sizeof(rx_buf));
    __HAL_DMA_DISABLE_IT(huart->hdmarx, DMA_IT_HT);
}
```

Acronimi:

- **HAL** = Hardware Abstraction Layer, libreria driver STMicroelectronics

Nota HAL: con parità abilitata e 8 bit dati, WordLength deve essere 9B perché la HAL conta il bit di parità nella lunghezza parola. È una fonte classica di bug.

ESP-IDF - UART

```
#include "driver/uart.h"

const uart_port_t port = UART_NUM_1;
uart_config_t cfg = {
    .baud_rate = 115200,
    .data_bits = UART_DATA_8_BITS,
    .parity = UART_PARITY_DISABLE,
    .stop_bits = UART_STOP_BITS_1,
    .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    .source_clk = UART_SCLK_DEFAULT,
};
ESP_ERROR_CHECK(uart_param_config(port, &cfg));
ESP_ERROR_CHECK(uart_set_pin(port, GPIO_NUM_17, GPIO_NUM_16,
    UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE)); // TX, RX, RTS, CTS
// buffer RX 1024 byte, TX 0 = trasmissione bloccante, nessuna event queue
ESP_ERROR_CHECK(uart_driver_install(port, 1024, 0, 0, NULL, 0));

uart_write_bytes(port, "OK\r\n", 4);

uint8_t buf[128];
int n = uart_read_bytes(port, buf, sizeof(buf), pdMS_TO_TICKS(100)); // n = byte letti
// Per RS-485: uart_set_mode(port, UART_MODE_RS485_HALF_DUPLEX) con pin RTS come DE
```

Acronimi:

- **ESP-IDF** = Espressif Internet-of-Things Development Framework

3. SPI

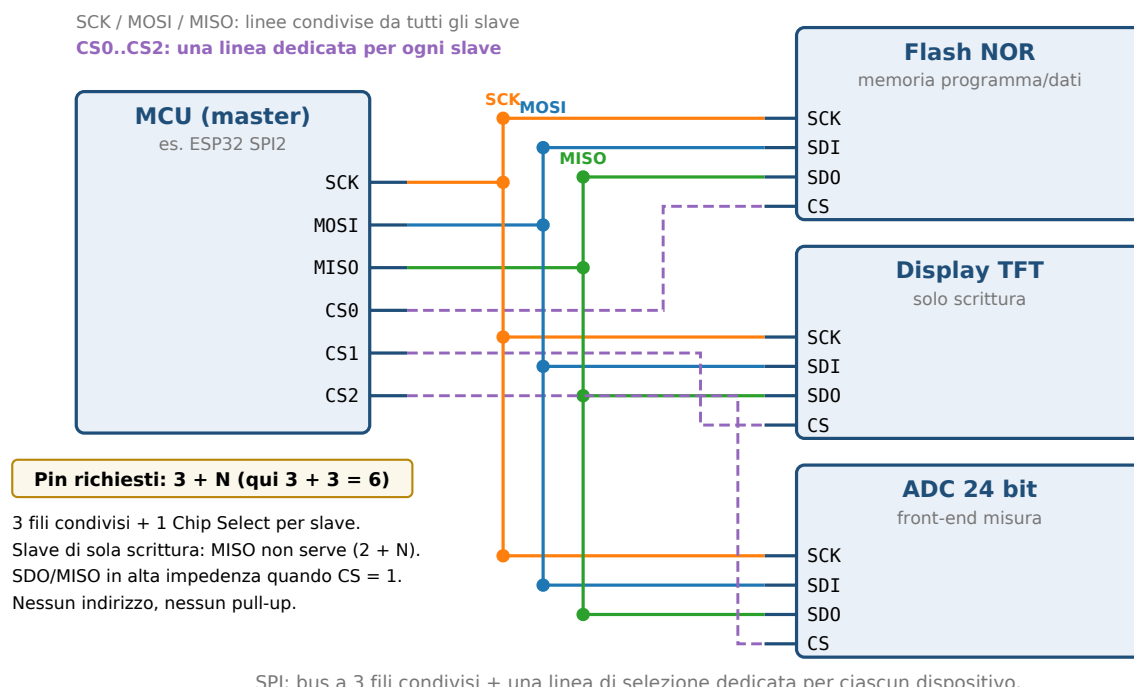
SPI = Serial Peripheral Interface, introdotto da Motorola negli anni '80. È un'interfaccia sincrona, full duplex, master-slave. **Non esiste uno standard formale**: ogni dispositivo definisce nel proprio datasheet modalità di clock, formato dei comandi e temporizzazioni. Questo lo rende velocissimo e semplice in hardware ma poco 'plug and play'.

3.1 Linee

Linea	Nome alternativo	Direzione	Funzione
SCK / SCLK	CLK	Master -> Slave	Clock generato sempre e solo dal master. Ogni impulso trasferisce 1 bit in ciascuna direzione.
MOSI	SDO (master), SDI (slave), COPI, D0	Master -> Slave	Dati dal master allo slave.
MISO	SDI (master), SDO (slave), CIPO, D1	Slave -> Master	Dati dallo slave al master. Deve essere in alta impedenza quando lo slave non è selezionato.
CS / SS / nCS	CE, STE	Master -> Slave	Chip Select, attivo basso. Uno per ogni slave. Delimita anche la transazione: molti dispositivi resettano la macchina a stati sul fronte di CS.

Acronimi:

- **SCK** = Serial Clock
- **SDO** = Serial Data Out
- **SDI** = Serial Data In
- **COPI** = Controller Out Peripheral In
- **CIPO** = Controller In Peripheral Out
- **nCS** = Chip Select negato (attivo basso)
- **CE** = Chip Enable
- **STE** = Slave Transmit Enable



SPI: bus a 3 fili condivisi + una linea di selezione dedicata per ciascun dispositivo.

Figura 3 - Bus SPI con tre slave: le linee di clock e dati sono condivise, ogni slave ha il proprio Chip Select.

3.2 Principio di funzionamento: due shift register

Il modello mentale corretto è un **anello di shift register**: il master ha un registro a 8 bit, lo slave anche. Ad ogni fronte di clock, il bit più significativo del master esce su MOSI ed entra nel registro

dello slave; contemporaneamente il bit dello slave esce su MISO ed entra nel master. Dopo 8 clock i due registri si sono scambiati il contenuto. Conseguenze fondamentali:

- SPI è **sempre full duplex**: non si può 'solo leggere'. Per leggere N byte il master deve trasmettere N byte (dummy, spesso 0x00 o 0xFF).
- Non esiste **acknowledge**: il master non sa se lo slave esiste o ha capito. La verifica va fatta a livello applicativo (es. leggere un registro WHO_AM_I).
- La velocità è limitata solo dal dispositivo più lento e dal layout: comunemente 1-50 MHz, oltre 100 MHz per memorie Quad-SPI.
- I dati sono tipicamente **MSB first**, ma alcuni dispositivi usano LSB first: va verificato sul datasheet.

Calcolo: throughput SPI e tempo di rinfresco di un display

In SPI non c'è overhead di start/stop/ack: 1 impulso di clock = 1 bit $\rightarrow$ byte/s = $f_{SCK} / 8$

A 10 MHz: $10\,000\,000 / 8 = \mathbf{1,25\ MB/s}$; a 40 MHz: **5 MB/s**. Confronto: UART a 115200 8N1 = 11,5 kB/s, I2C Fast-mode ~40 kB/s.

Frame di un display 320 x 240 a 16 bit/pixel: $320 \times 240 \times 2 = \mathbf{153\,600\ byte}$

Tempo per frame a 40 MHz: $153\,600 \times 8 / 40\,000\,000 = \mathbf{30,7\ ms}$ $\rightarrow$ max ~32 fps (senza contare comandi e gap tra transazioni).

Lo stesso frame su I2C a 400 kHz richiederebbe ~4 s: ecco perché i display usano SPI (o interfacce parallele).

Frequenza SPI su STM32 = $f_{PCLK} / \text{prescaler}$, con prescaler in potenze di 2 (2..256): con PCLK 84 MHz i valori ottenibili sono 42, 21, 10,5, 5,25 MHz... Non si può impostare un valore arbitrario: si sceglie il più alto sotto il massimo del dispositivo.

Acronimi:

- **fps** = frame per secondo
- **PCLK** = Peripheral Clock, clock del bus periferico

3.3 Modalità di clock: CPOL e CPHA

Questa è la domanda SPI più frequente al colloquio. Due parametri definiscono quando i dati vengono campionati:

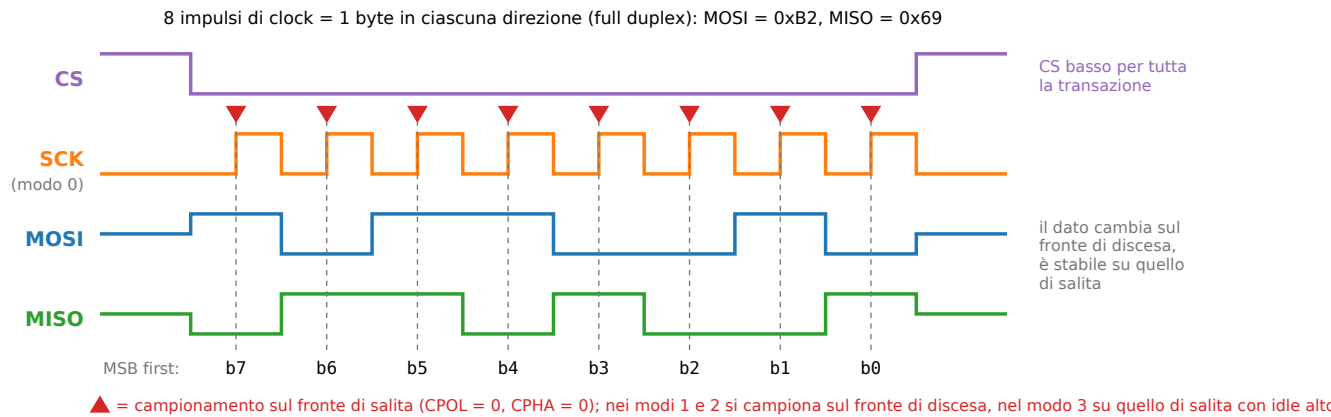
Acronimi:

- **CPOL** = Clock Polarity, polarità del clock a riposo
- **CPHA** = Clock Phase, fase di campionamento
- **CPOL** (Clock Polarity): livello del clock a riposo. CPOL=0 $\rightarrow$ idle basso; CPOL=1 $\rightarrow$ idle alto.
- **CPHA** (Clock Phase): CPHA=0 $\rightarrow$ il dato è campionato sul **primo** fronte del clock (e cambia sul secondo); CPHA=1 $\rightarrow$ campionato sul **secondo** fronte (e cambia sul primo).

Modo	CPOL	CPHA	Idle SCK	Campionamento	Dispositivi tipici
0	0	0	Basso	Fronte di salita	Il più diffuso: SD card, molte flash, display, sensori
1	0	1	Basso	Fronte di discesa	Alcuni ADC/DAC
2	1	0	Alto	Fronte di discesa	Raro
3	1	1	Alto	Fronte di salita	Molte flash e sensori (spesso compatibili anche con modo 0)

Acronimi:

- **SD** = Secure Digital, scheda di memoria
- **ADC** = Analog-to-Digital Converter, convertitore analogico-digitale
- **DAC** = Digital-to-Analog Converter



Transazione SPI in modo 0: la transazione è delimitata da CS; il master genera il clock e scambia un byte con lo slave.

Figura 4 - Scambio di un byte in modo 0: 8 fronti di salita = 8 bit per direzione, CS delimita la transazione.

Con CPHA=0 il primo bit deve essere già valido su MOSI/MISO quando CS si abbassa, prima del primo fronte di clock. Quando un dispositivo SPI 'non risponde' la prima verifica è la modalità di clock, la seconda l'ordine dei bit, la terza la gestione del CS.

3.4 Topologie multi-slave

- **Chip Select indipendenti:** soluzione standard. Costa un GPIO (General Purpose Input/Output, pin digitale generico) per slave; si può usare un decoder/demultiplexer per risparmiare pin.
- **Daisy chain:** MISO di uno slave va al MOSI del successivo formando un unico lungo shift register; un solo CS. Usata da driver LED (Light-Emitting Diode), shift register (74HC595), alcuni ADC. I dati per l'ultimo dispositivo vanno trasmessi per primi.
- Con CS indipendenti gli slave condividono SCK/MOSI/MISO: tutti i MISO devono andare in tri-state quando non selezionati, altrimenti un dispositivo mal progettato blocca il bus.

3.5 Varianti

- **3-wire / half duplex:** MOSI e MISO uniti in una sola linea bidirezionale (es. alcuni display, sensori Bosch in modalità SPI 3 fili).
- **Dual / Quad SPI (QSPI):** 2 o 4 linee dati bidirezionali; usate per flash NOR esterne. Permettono l'**XIP** (execute in place: la CPU esegue codice direttamente dalla flash esterna, come su ESP32).
- **Octal SPI, HyperBus:** 8 linee, per RAM/flash ad alta banda.
- **TI SSI (Synchronous Serial Interface (Texas Instruments)), Microwire:** varianti storiche compatibili in alcune periferiche STM32.

Acronimi:

- **QSPI** = Quad Serial Peripheral Interface, SPI a 4 linee dati
- **NOR** = flash a celle NOR, accesso casuale, adatta al codice
- **CPU** = Central Processing Unit
- **RAM** = Random Access Memory

3.6 Quando si usa

- **Memorie:** flash NOR/NAND, EEPROM veloci, FRAM, SD card (modo SPI).
- **Display:** TFT (ILI9341, ST7789), e-paper, OLED, dove servono decine di MHz per rinfrescare frame.
- **Convertitori:** ADC/DAC a elevato sample rate (es. front-end di misura in energy metering come ADE7xxx/ATM90Exx, che espongono i registri di energia via SPI).
- **Comunicazione:** controller Ethernet (W5500, ENC28J60), radio (nRF24L01, SX1276 LoRa, CC1101), CAN controller esterni (MCP2515).

- **Sensori** quando serve alta banda (IMU a kHz di ODR) o quando l'I2C è già saturo.

Acronimi:

- **NAND** = flash a celle NAND, alta densità, accesso a pagine
- **EEPROM** = Electrically Erasable Programmable Read-Only Memory
- **FRAM** = Ferroelectric Random Access Memory
- **TFT** = Thin-Film Transistor, tecnologia di display LCD
- **OLED** = Organic Light-Emitting Diode
- **LoRa** = Long Range, modulazione radio a lungo raggio
- **IMU** = Inertial Measurement Unit, unità inerziale (accelerometro + giroscopio)
- **ODR** = Output Data Rate, frequenza di aggiornamento dei campioni

Limiti: molti pin con tanti slave, nessun ack, nessuno standard sul formato, non adatto a distanze oltre pochi decimetri (segnali single-ended veloci), un solo master.

3.7 Esempi di codice

STM32 HAL - SPI

```
// CubeMX: SPI1 master, 8 bit, MSB first, CPOL low / CPHA 1 edge (modo 0), prescaler -> es. 10 MHz
// CS gestito via GPIO software (più flessibile del NSS hardware)
#define CS_LOW()   HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET)
#define CS_HIGH()  HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET)

// Lettura registro di un sensore: byte 1 = comando (bit7=1 -> read), byte 2 = dummy per ricevere il dato
uint8_t spi_read_reg(uint8_t reg)
{
    uint8_t tx[2] = { reg | 0x80, 0x00 };
    uint8_t rx[2] = { 0 };
    CS_LOW();
    HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, 100); // full duplex: 2 byte out, 2 byte in
    CS_HIGH();
    return rx[1]; // rx[0] è spazzatura (lo slave sta ancora ricevendo il comando)
}

// Scrittura a blocchi con DMA (non bloccante); il CS va alzato nel callback
HAL_SPI_Transmit_DMA(&hspi1, frame_buf, frame_len);
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi) { CS_HIGH(); }
```

ESP-IDF - SPI master

```
#include "driver/spi_master.h"

spi_bus_config_t bus = {
    .mosi_io_num = 23, .miso_io_num = 19, .sclk_io_num = 18,
    .quadwp_io_num = -1, .quadhd_io_num = -1,
    .max_transfer_sz = 4096,
};
ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &bus, SPI_DMA_CH_AUTO));

spi_device_interface_config_t dev = {
    .clock_speed_hz = 10 * 1000 * 1000, // 10 MHz
    .mode            = 0,                 // CPOL=0, CPHA=0
    .spics_io_num    = 5,                 // CS gestito dal driver
    .queue_size      = 4,
};
spi_device_handle_t h;
ESP_ERROR_CHECK(spi_bus_add_device(SPI2_HOST, &dev, &h));

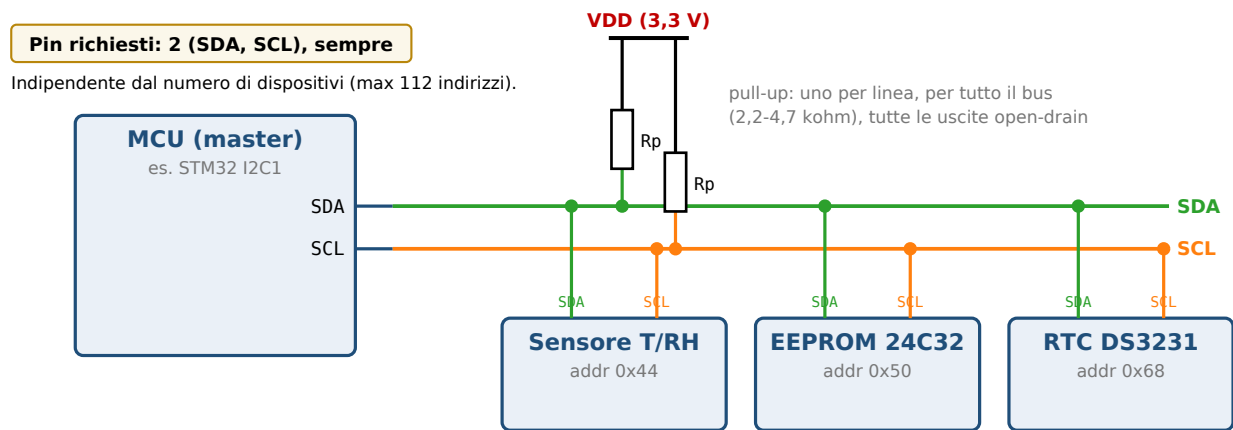
uint8_t tx[2] = { 0x80 | 0x0F, 0x00 }, rx[2];
spi_transaction_t t = {
    .length      = 8 * sizeof(tx), // in BIT
    .tx_buffer   = tx,
    .rx_buffer   = rx,
};
ESP_ERROR_CHECK(spi_device_transmit(h, &t)); // bloccante; spi_device_queue_trans() per async
// Nota: su ESP32 SPI0/SPI1 sono riservati alla flash (XIP); usare SPI2/SPI3 (HSPI/VSPI)
```

4. I2C

I2C (Inter-Integrated Circuit, pronunciato 'I-squared-C', talvolta scritto IIC o I²C) è stato sviluppato da Philips (oggi NXP) nel 1982. È un bus sincrono, half duplex, multi-master e multi-slave che usa **solo due fili** condivisi tra tutti i dispositivi. A differenza di SPI esiste una **specificazione ufficiale**: NXP UM10204 (I2C-bus specification and user manual), oggi alla revisione 7.

4.1 Linee e livello elettrico: open-drain e pull-up

- **SDA** (Serial Data) e **SCL** (Serial Clock), più GND.
- Le uscite di tutti i dispositivi sono **open-drain**: possono solo tirare la linea a 0 o lasciarla libera. Il livello alto è fornito da **resistenze di pull-up** esterne (una per linea). Il bus è un AND cablato: basta che un dispositivo tiri a 0 e la linea è a 0.
- Questo consente **clock stretching**, **arbitraggio multi-master** e connessione di dispositivi a tensioni diverse (con level shifter bidirezionali, es. PCA9306) senza corto circuiti.



I2C: tutti i dispositivi in parallelo sulle stesse due linee; ognuno risponde solo al proprio indirizzo a 7 bit.
Figura 5 - Bus I2C: due linee per tutti i dispositivi, con un pull-up per linea verso VDD.

Acronimi:

- **VDD** = tensione di alimentazione positiva

Dimensionamento dei pull-up: il valore minimo è fissato dalla corrente massima che un'uscita può assorbire (3 mA in Standard/Fast mode, V_{OL} max 0,4 V): $R_{min} = (V_{DD} - 0,4 \text{ V}) / 3 \text{ mA}$, circa 1 kΩ a 3,3 V. Il valore massimo è fissato dal **tempo di salita** ammesso, che dipende dalla capacità del bus C_b (max 400 pF): $R_{max} = t_r / (0,8473 \times C_b)$. Valori pratici: 4,7 kΩ a 100 kHz, 2,2 kΩ a 400 kHz, 1 kΩ o pull-up attivi a 1 MHz. Pull-up troppo alti -> fronti lenti -> errori a velocità alte; troppo bassi -> consumo e violazione di V_{OL} . I pull-up interni degli MCU (30-50 kΩ) bastano solo per prototipi a bassa velocità con pochi cm di pista.

Calcolo: dimensionamento dei pull-up I2C ($V_{DD} = 3,3 \text{ V}$)

Limite inferiore (corrente di sink): quando un dispositivo tira la linea a 0 la corrente nel pull-up non deve superare $I_{OL} = 3 \text{ mA}$ garantendo $V_{OL} \leq 0,4 \text{ V}$

$$R_{\min} = (V_{DD} - V_{OL}) / I_{OL} = (3,3 - 0,4) / 0,003 = \mathbf{967 \, \Omega} \rightarrow \text{in pratica non meno di } 1 \text{ k}\Omega$$

Limite superiore (tempo di salita): il fronte di salita è la carica di C_b attraverso R . La specifica definisce t_r tra il 30 % e il 70 % di V_{DD} : dalla legge esponenziale RC, $t_r = R \times C_b \times [\ln(1 - 0,3) - \ln(1 - 0,7)] = R \times C_b \times \ln(0,7/0,3) = \mathbf{0,8473 \times R \times C_b}$

$$R_{\max} = t_{r,\max} / (0,8473 \times C_b)$$

Fast-mode ($t_r \leq 300 \text{ ns}$), bus da 100 pF : $R_{\max} = 300\text{e-}9 / (0,8473 \times 100\text{e-}12) = \mathbf{3,54 \text{ k}\Omega} \rightarrow$ si sceglie $2,2\text{-}3,3 \text{ k}\Omega$

Standard-mode ($t_r \leq 1000 \text{ ns}$), bus al massimo di 400 pF : $R_{\max} = 1000\text{e-}9 / (0,8473 \times 400\text{e-}12) = \mathbf{2,95 \text{ k}\Omega}$

Standard-mode, bus corto da 50 pF : $R_{\max} = 1000\text{e-}9 / (0,8473 \times 50\text{e-}12) = \mathbf{23,6 \text{ k}\Omega} \rightarrow$ qui $4,7$ o $10 \text{ k}\Omega$ vanno benissimo

Consumo: con la linea a 0 il pull-up dissipa $I = V_{DD} / R = 3,3 / 2200 = 1,5 \text{ mA}$ per linea. Con SCL a 400 kHz la linea è bassa $\sim 50 \%$ del tempo $\rightarrow \sim 0,75 \text{ mA}$ medi per linea durante il traffico: rilevante nei dispositivi a batteria.

Stima di C_b : $\sim 10 \text{ pF}$ per ogni pin di dispositivo + $\sim 1 \text{ pF/cm}$ di pista PCB ($0,5\text{-}1 \text{ pF/cm}$; i cavi piatti arrivano a $50\text{-}100 \text{ pF/m}$).

Acronimi:

- **PCB** = Printed Circuit Board, circuito stampato

4.2 Velocità

Modo	Velocità max	Note
Standard-mode (Sm)	100 kbit/s	Il più compatibile.
Fast-mode (Fm)	400 kbit/s	Il più usato oggi. Retrocompatibile con Sm.
Fast-mode Plus (Fm+)	1 Mbit/s	Driver più forti (20 mA), capacità bus fino a 550 pF.
High-speed mode (Hs)	3,4 Mbit/s	Richiede master code e current source per il pull-up; raro nei MCU general purpose.
Ultra Fast-mode (UFm)	5 Mbit/s	Unidirezionale, push-pull, nessun ack: di fatto un altro protocollo.

La velocità netta è inferiore a quella nominale: ogni byte richiede 9 clock (8 dati + ACK) più start/stop e indirizzo. Con Fast-mode, leggere 6 byte da un registro richiede circa $200 \, \mu\text{s}$.

Acronimi:

- **ACK** = Acknowledge, conferma di ricezione

Calcolo: durata di una lettura di 6 byte da un registro (Fast-mode, 400 kHz)

Periodo di clock: $T_{SCL} = 1 / 400\,000 = \mathbf{2,5 \, \mu\text{s}}$

Ogni byte sul bus occupa 8 bit + 1 ACK = **9 clock** = $22,5 \, \mu\text{s}$

Sequenza write-then-read: S + [ADDR+W] + [REG] + Sr + [ADDR+R] + 6 x [DATA] + P

Byte totali sul bus = 1 (indirizzo) + 1 (registro) + 1 (indirizzo) + 6 (dati) = 9 byte $\rightarrow 9 \times 9 = \mathbf{81 \text{ clock}}$

Condizioni S, Sr, P: circa 1 clock ciascuna ($t_{HD;STA}$, $t_{SU;STO}$, t_{BUF}) $\rightarrow \sim 3 \text{ clock}$

Totale $\sim 84 \text{ clock} \times 2,5 \, \mu\text{s} = \mathbf{\sim 210 \, \mu\text{s}}$, senza clock stretching

Byte utili / byte sul bus = $6 / 9 = 67 \%$; contando anche gli ACK: $48 \text{ bit} / 84 \text{ clock} = \mathbf{57 \% \text{ di efficienza}}$ $\rightarrow$ throughput netto $\sim 29 \text{ kB/s}$ a 400 kHz per letture corte, $\sim 40 \text{ kB/s}$ ($8/9 \times 50 \text{ kB/s}$) per blocchi lunghi

La stessa lettura in Standard-mode (100 kHz) dura 4 volte tanto: **$\sim 840 \, \mu\text{s}$** . Con 10 sensori campionati a 100 Hz il bus Sm è già occupato all'84 %.

4.3 Il protocollo bit per bit

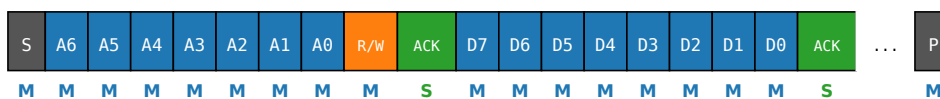
- **Idle:** SDA e SCL alti (tirati dai pull-up).
- **START (S):** SDA passa da alto a basso **mentre SCL è alto**. Solo il master genera START.
- **Bit di dato:** SDA può cambiare solo mentre SCL è basso; viene campionato quando SCL è alto. È l'inverso della regola dello START/STOP, che per questo sono riconoscibili come condizioni speciali.
- **Indirizzo:** 7 bit MSB first seguiti dal bit **R/W** (0 = write, 1 = read). Il byte di indirizzo è quindi ($\text{addr} \ll 1$) | RW. Esiste anche l'indirizzamento a 10 bit (prefisso 11110xx).
- **ACK/NACK:** al 9° clock il trasmettitore rilascia SDA e il ricevitore la tira a 0 per confermare (**ACK**). Se resta alta (**NACK**): nessun dispositivo a quell'indirizzo, dispositivo occupato, o - in lettura - il master segnala 'ultimo byte'.
- **STOP (P):** SDA passa da basso ad alto mentre SCL è alto. Libera il bus.
- **Repeated START (Sr):** un nuovo START senza STOP intermedio. Serve per cambiare direzione (write poi read) mantenendo il possesso del bus: essenziale in multi-master per garantire atomicità.
- **Clock stretching:** uno slave può tenere SCL basso per rallentare il master quando non è pronto (es. sensore in conversione). Il master deve rilevarlo (i controller hardware lo fanno; le implementazioni bit-bang spesso no). Alcuni dispositivi non lo supportano (es. Raspberry Pi storicamente aveva bug su questo).

Acronimi:

- **R/W** = Read/Write, bit di direzione
- **NACK** = Not Acknowledge, mancata conferma

Ogni byte = 9 clock SCL (8 dati + ACK). SDA cambia solo con SCL basso; START e STOP sono le uniche transizioni di SDA con SCL alto.

byte 1: indirizzo 7 bit + R/W (= $\text{addr} \ll 1$ | RW) byte 2: dato / puntatore registro



M = pilota il master, S = pilota lo slave (ACK: il ricevitore tira SDA a 0 durante il 9° clock)

S = START (SDA 1->0 con SCL = 1) P = STOP (SDA 0->1 con SCL = 1) Sr = repeated START (cambia direzione senza rilasciare il bus)

Struttura di una transazione I2C: scrittura di un byte a un dispositivo (R/W = 0).

Figura 6 - Anatomia di una transazione I2C: ogni byte è seguito da un bit di ACK pilotato dal ricevitore.

4.4 La transazione tipica: lettura di un registro

Sequenza logica write-then-read (combined format)

Master:	S		ADDR+W		REG_PTR		Sr		ADDR+R				P
Slave :			ACK		ACK				ACK		DATA0		DATA1
Master:											ACK		NACK

1. START, indirizzo con W: il master dice "voglio parlare con te"
2. Scrive il puntatore al registro interno da leggere
3. Repeated START, stesso indirizzo con R: cambia direzione senza rilasciare il bus
4. Lo slave trasmette i byte a partire dal registro puntato (auto-incremento)
5. Il master fa ACK su ogni byte tranne l'ultimo (NACK) e poi STOP

Questa è la sequenza implementata da `HAL_I2C_Mem_Read` e da `i2c_master_transmit_receive`. Quasi tutti i sensori seguono questo modello 'register map': leggere il datasheet per capire se il puntatore auto-incrementa e quale bit lo abilita (alcuni dispositivi Bosch/ST richiedono un bit specifico per il multi-byte read).

4.5 Multi-master e arbitraggio

Grazie all'open-drain, due master che trasmettono contemporaneamente non si danneggiano. Ogni master, mentre trasmette, **rilegge SDA**: se ha rilasciato la linea (voleva trasmettere 1) ma la vede a 0, significa che un altro master sta trasmettendo 0 e ha vinto l'arbitraggio; il perdente si ritira immediatamente e riproverà dopo lo STOP. Il meccanismo è non distruttivo e senza perdita di dati. È lo stesso principio di CAN. Nella pratica il multi-master si usa raramente perché complica la gestione degli errori.

4.6 Indirizzi, conflitti e problemi tipici

- Alcuni indirizzi sono **riservati**: 0000 000 (general call), 0000 001 (START byte), 0000 1xx (Hs master code), 1111 0xx (10 bit), 1111 1xx (device ID). Restano 112 indirizzi utili.
- **Conflitto di indirizzi**: due dispositivi identici sullo stesso bus. Soluzioni: pin ADDR sul dispositivo (spesso 1-2 bit configurabili), multiplexer I2C (TCA9548A), un secondo bus.
- **Attenzione alla notazione**: i datasheet indicano l'indirizzo talvolta a 7 bit (0x68) e talvolta già a 8 bit con R/W (0xD0 / 0xD1). La HAL STM32 vuole l'indirizzo a 8 bit (shiftato), ESP-IDF e Linux quello a 7 bit. È il bug I2C più frequente in assoluto.
- **Bus bloccato (SDA a 0)**: se il master viene resettato a metà di una lettura, lo slave può restare con SDA bassa aspettando clock. Nessuno START è possibile. **Recovery**: generare fino a 9 impulsi su SCL con GPIO finché SDA torna alta, poi uno STOP. Da implementare sempre nei prodotti industriali; molti MCU offrono anche un bus reset nella periferica.
- **Scansione del bus**: inviare START+ADDR a tutti i 112 indirizzi e vedere chi fa ACK (`HAL_I2C_IsDeviceReady`, `i2c_master_probe`, `i2cdetect` su Linux).

Acronimi:

- **ID** = Identifier, identificatore

4.7 SMBus e PMBus

SMBus (System Management Bus, Intel) è un sottoinsieme di I2C con regole più strette: timeout obbligatorio di 35 ms (evita il bus lock), livelli logici fissi, PEC (CRC-8) opzionale, comandi standardizzati, linea ALERT. **PMBus** aggiunge un set di comandi per alimentatori digitali. Dispositivi SMBus funzionano di norma su un master I2C, ma non sempre il contrario.

Acronimi:

- **SMBus** = System Management Bus
- **PMBus** = Power Management Bus
- **PEC** = Packet Error Code, CRC-8 opzionale di SMBus

4.8 Quando si usa

- **Sensori a bassa velocità**: temperatura/umidità (SHT4x, BME280), pressione, luce, IMU a basso ODR, magnetometri.
- **Memorie e RTC** (Real-Time Clock, orologio in tempo reale): EEPROM (24Cxx), RTC (DS3231, PCF8563), spesso con batteria tampone.
- **Gestione di sistema**: PMIC (Power Management Integrated Circuit), battery gauge, monitor di tensione/corrente (INA219/INA226), sensori di temperatura di scheda, ventole.
- **Espansione I/O**: GPIO expander (PCA9555, MCP23017), driver LED/PWM (PCA9685), tastiere capacitive.
- **Display** piccoli: OLED SSD1306, LCD (Liquid Crystal Display) con backpack PCF8574.
- **Configurazione** di altri chip: codec audio, PHY Ethernet (MDIO è simile), tuner, EDID dei monitor (DDC è I2C).

Acronimi:

- **I/O** = Input/Output
- **PWM** = Pulse-Width Modulation
- **PHY** = Physical layer transceiver, chip di livello fisico
- **MDIO** = Management Data Input/Output
- **EDID** = Extended Display Identification Data
- **DDC** = Display Data Channel

Limiti: velocità, distanza (capacità 400 pF -> tipicamente meno di 1 m; esistono bus extender come P82B715 per arrivare a decine di metri), 112 indirizzi, sensibile al rumore sui fronti (glitch filter), gestione errori delicata.

4.9 Esempi di codice

STM32 HAL - I2C master

```
// CubeMX: I2C1, Fast mode 400 kHz, 7-bit addressing
#define MPU_ADDR_7BIT    0x68
#define MPU_ADDR        (MPU_ADDR_7BIT << 1)    // la HAL vuole l'indirizzo a 8 bit!

// Verifica presenza (scan di un indirizzo): 3 tentativi, timeout 10 ms
if (HAL_I2C_IsDeviceReady(&hi2c1, MPU_ADDR, 3, 10) != HAL_OK) { /* NACK: assente */ }

// Lettura di 6 byte a partire dal registro 0x3B: genera S, ADDR+W, 0x3B, Sr, ADDR+R, ... P
uint8_t acc[6];
HAL_StatusTypeDef st = HAL_I2C_Mem_Read(&hi2c1, MPU_ADDR, 0x3B,
                                         I2C_MEMADD_SIZE_8BIT, acc, sizeof(acc), 100);

if (st != HAL_OK) {
    uint32_t err = HAL_I2C_GetError(&hi2c1);    // HAL_I2C_ERROR_AF = NACK ricevuto
}

// Scrittura di un registro
uint8_t val = 0x01;
HAL_I2C_Mem_Write(&hi2c1, MPU_ADDR, 0x6B, I2C_MEMADD_SIZE_8BIT, &val, 1, 100);

// Versioni non bloccanti: HAL_I2C_Mem_Read_IT / _DMA + HAL_I2C_MemRxCpltCallback
```

ESP-IDF - I2C master (driver i2c_master)

```
#include "driver/i2c_master.h"    // nuovo driver (ESP-IDF >= 5.2); il legacy è driver/i2c.h

i2c_master_bus_config_t bus_cfg = {
    .i2c_port    = I2C_NUM_0,
    .sda_io_num  = GPIO_NUM_21,
    .scl_io_num  = GPIO_NUM_22,
    .clk_source  = I2C_CLK_SRC_DEFAULT,
    .glitch_ignore_cnt = 7,
    .flags.enable_internal_pullup = true,    // solo per prototipi: usare pull-up esterni
};
i2c_master_bus_handle_t bus;
ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));

i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7,
    .device_address  = 0x68,                // indirizzo a 7 bit (NON shiftato)
    .scl_speed_hz    = 400000,
};
i2c_master_dev_handle_t dev;
ESP_ERROR_CHECK(i2c_master_bus_add_device(bus, &dev_cfg, &dev));

// Probe: ESP_OK se il dispositivo fa ACK
esp_err_t r = i2c_master_probe(bus, 0x68, 50);

// write-then-read con repeated start: reg 0x3B, 6 byte, timeout 100 ms
uint8_t reg = 0x3B, acc[6];
ESP_ERROR_CHECK(i2c_master_transmit_receive(dev, &reg, 1, acc, sizeof(acc), 100));
```

5. Tabella comparativa e criteri di scelta

Caratteristica	UART	SPI	I2C
Sincronizzazione	Asincrona (baud rate concordato, start bit)	Sincrona (SCK dal master)	Sincrona (SCL dal master, con clock stretching)
Numero di fili (oltre GND)	2 (TX, RX); +2 con RTS/CTS	3 + 1 CS per slave (SCK, MOSI, MISO, CSn)	2 (SDA, SCL) per qualsiasi numero di dispositivi
Duplex	Full	Full (sempre, per costruzione)	Half
Topologia	Punto-punto (bus solo con RS-485/multidrop)	Un master, N slave selezionati via hardware	Multi-master, multi-slave, indirizzamento software
Indirizzamento	Nessuno (a livello applicativo, es. Modbus)	Hardware (Chip Select)	7 bit (112 utili) o 10 bit
Velocità tipica	9600 - 921600 bit/s (fino a qualche Mbit/s)	1 - 50 MHz; >100 MHz Quad/Octal	100 kHz, 400 kHz, 1 MHz (3,4 MHz Hs)
Acknowledge	No	No	Sì, per ogni byte
Rilevamento errori	Parità, framing, overrun	Nessuno nativo	ACK/NACK; PEC in SMBus
Ordine dei bit	LSB first	MSB first (di norma)	MSB first
Uscite	Push-pull	Push-pull (MISO tri-state)	Open-drain + pull-up
Standard	De facto; elettrici RS-232/422/485	Nessuno (Motorola de facto)	NXP UM10204
Consumo	Basso	Alto a velocità elevate	Basso, ma i pull-up dissipano quando la linea è a 0
Distanza	Metri (TTL), 15 m RS-232, 1200 m RS-485	Centimetri	Meno di 1 m (400 pF); estensori fino a decine di m
Complessità firmware	Bassa; framing applicativo da gestire	Bassa; attenzione a modi e CS	Media; gestione NACK, timeout, bus recovery
Uso tipico	Debug, GPS, modem, bootloader, bus di campo	Flash, display, ADC veloci, SD, Ethernet/radio	Sensori, EEPROM, RTC, PMIC, I/O expander

5.0 Quanti pin servono?

Riepilogo del costo in pin dell'MCU per collegare N dispositivi, il criterio pratico che più spesso decide la scelta su schede piccole:

Interfaccia	Pin MCU per N dispositivi	N = 1	N = 4	N = 8	Note
UART	2 x N (TX, RX per ogni link)	2	8	16	Ogni dispositivo richiede una USART dedicata: i pin finiscono presto, ma anche le periferiche.
UART + RS-485	3 (TX, RX, DE)	3	3	3	Una sola USART per tutti i nodi grazie al bus multidrop; l'indirizzamento lo fa il protocollo (Modbus).
SPI	3 + N (SCK, MOSI, MISO, CSn)	4	7	11	2 + N con slave di sola scrittura. Con un decoder 3-a-8 sui CS: 3 + 3 = 6 per 8 slave.
I2C	2 (SDA, SCL)	2	2	2	Fino a 112 indirizzi; con dispositivi identici servono pin ADDR o un multiplexer (che a sua volta è un dispositivo I2C).
CAN	2 (CAN_TX, CAN_RX)	2	2	2	Indipendente dal numero di nodi; serve il transceiver esterno.
QSPI	6 (CLK, CS, IO0-IO3)	6	-	-	Tipicamente un solo dispositivo (flash/PSRAM) per periferica.

Acronimi:

- **PSRAM** = Pseudo-Static Random Access Memory

A questi vanno aggiunti GND (sempre) e, per dispositivi esterni alla scheda, eventuali pin di interrupt/data-ready (uno per sensore, indipendentemente dal bus) e di reset. Un sensore I2C 'da 2 pin' con linea INT costa in realtà 3 pin.

5.1 Come scegliere

- **Serve banda** (display, flash, streaming ADC)? -> **SPI**. Se serve XIP o >50 MHz -> Quad/Octal SPI.
- **Molti dispositivi lenti e pochi pin** (sensori, RTC, EEPROM)? -> **I2C**. Se il consumo in sleep è critico, ricordare i pull-up.
- **Due soli nodi, magari a distanza o verso un PC**? -> **UART**, con transceiver RS-485 per bus industriali (Modbus RTU) o RS-232 per legacy.
- **Bus multi-nodo robusto con arbitraggio e rilevamento errori in hardware** (automotive, industriale)? -> **CAN**.
- Spesso il dispositivo stesso impone la scelta: molti sensori offrono sia SPI che I2C (BME280, LSM6DS3, ADXL345) e si sceglie in base ai pin disponibili e al numero di dispositivi.

Trade-off da citare al colloquio

I2C risparmia pin e semplifica il layout, ma paga in velocità, in robustezza (bus lock, sensibilità ai fronti) e in complessità di gestione degli errori. SPI è veloce e deterministico ma senza ack, senza standard e costoso in pin. UART è universale e semplice, ma punto-punto e con overhead del 20%. Nessuno è 'migliore': si sceglie per banda, numero di dispositivi, pin disponibili, distanza, consumo e - non ultimo - per cosa supporta il chip che si deve usare.

5.2 Casi d'uso concreti: quale protocollo e perché

Otto situazioni reali di progetto. Per ciascuna: cosa serve, quale interfaccia si sceglie e, soprattutto, perché le alternative sarebbero peggiori. È il tipo di ragionamento che al colloquio conta più della definizione: dimostra che la scelta è motivata da banda, pin, distanza, consumo e da cosa offrono i componenti disponibili.

Caso 1 - Contatore di energia trifase con front-end di misura dedicato

Situazione. Una scheda con MCU STM32 legge un chip di misura (es. ATM90E32, ADE7880) che campiona tensioni e correnti a qualche kHz e calcola RMS, potenze ed energie in registri interni. La scheda deve poi esporre i dati a un sistema di supervisione a decine o centinaia di metri, in un quadro elettrico rumoroso, e conservare i coefficienti di calibrazione.

- **MCU - chip di misura: SPI.** I registri sono a 16/32 bit e vanno letti decine di volte al secondo con temporizzazione deterministica; a 10 MHz una lettura di 4 byte dura circa 3 μ s (32 bit / 10 MHz) e non disturba il campionamento. Il full duplex permette di inviare l'indirizzo del registro e ricevere il dato nella stessa transazione, e il pin di interrupt del chip (fine ciclo di misura) si abbina naturalmente a una transazione SPI in DMA.
- **Scheda - supervisione: UART + RS-485 con Modbus RTU.** È lo standard de facto di PLC, inverter e sistemi di building automation: il cliente si aspetta una register map Modbus. Il doppiopino differenziale copre 1200 m con immunità ai disturbi di modo comune, e su un solo bus si mettono fino a 32 contatori indirizzati via software.
- **Calibrazione e orologio: I2C.** Pochi byte scritti una volta in fabbrica (EEPROM) e un RTC per le curve di carico oraria: traffico irrisorio, e i due dispositivi condividono gli stessi 2 pin.
- **Console di fabbrica e bootloader: UART.** Programmazione e collaudo su banco con un convertitore USB-UART da pochi euro.

Perché non le alternative:

- I2C per il front-end di misura: la lettura di un blocco di 20 registri a 400 kHz durerebbe ~1 ms e sarebbe soggetta a clock stretching; molti chip di misura non lo offrono neppure, o lo limitano a 100 kHz.
- SPI verso il quadro: segnali single-ended a pochi MHz non superano qualche decina di centimetri; nessun transceiver SPI standard per bus multidrop.
- CAN verso la supervisione: tecnicamente ottimo, ma il mondo dell'energy metering e dei PLC parla Modbus; un contatore CAN richiederebbe un gateway e un protocollo applicativo (CANopen) che il cliente non ha.

Acronimi:

- **RMS** = Root Mean Square, valore efficace

Caso 2 - Sensore ambientale IoT a batteria con display e-paper

Situazione. Un ESP32 in deep sleep si sveglia ogni 5 minuti, legge temperatura, umidità e pressione, aggiorna un display e-paper da 2,9 pollici e trasmette via Wi-Fi. Deve durare un anno con due batterie AA.

- **Sensori: I2C.** Un BME280 e un sensore di luce ambientale costano 2 pin in totale, hanno indirizzi diversi e producono pochi byte per lettura. La velocità è irrilevante: 8 byte a 400 kHz sono 200 μ s.
- **Display e-paper: SPI.** Un frame monocromatico 296 x 128 pixel è 4736 byte; a 10 MHz si trasferisce in circa 4 ms, poi il display si aggiorna da solo e si può togliere alimentazione. Il display richiede comunque pin aggiuntivi (DC, RST, BUSY) che nessun protocollo elimina.

Perché non le alternative:

- SPI per i sensori: 3 + 2 = 5 pin invece di 2, e il BME280 in modalità SPI perde il vantaggio di condividere il bus con altri sensori. I pin su un modulo ESP32 sono la risorsa più scarsa.
- I2C per il display: pochi e-paper offrono I2C, e il frame richiederebbe ~120 ms a 400 kHz: 30 volte più tempo con radio e CPU accese, cioè consumo. Su un dispositivo a batteria il tempo

attivo è tutto.

- Attenzione ai pull-up I2C: con SDA/SCL a riposo alti non consumano, ma se un sensore tiene SDA bassa dopo un errore dissipano ~1,5 mA in continuo: il recovery del bus va gestito prima di tornare in sleep.

Caso 3 - Data logger veicolare con GPS e scheda SD

Situazione. Registrare posizione, velocità e alcuni segnali analogici a 10 Hz su microSD per settimane, con un MCU STM32.

- **Modulo GPS: UART.** I moduli GPS espongono UART con frasi NMEA testuali a 9600 o 115200 baud, 1-10 volte al secondo: flusso lento, punto-punto, senza bisogno di clock. Si riceve con DMA circolare e parsing sull'IDLE.
- **microSD: SPI.** A 20-25 MHz si scrivono blocchi da 512 byte in ~200 µs; il file system (FatFs) lavora sopra un driver SPI standard. Le SD supportano anche il bus SDIO a 4 bit, più veloce, ma per 10 Hz di log non serve.
- **Segnali analogici: ADC interno** dell'MCU, senza bus.

Perché non le alternative:

- I2C per il GPS: alcuni moduli u-blox lo offrono (DDC), ma il protocollo binario UBX via I2C è meno documentato e l'UART è sempre presente e già configurato in fabbrica.
- I2C per la SD: non esiste. UART per la SD: non esiste. La SD parla SPI o SDIO, e il componente decide.
- USB come storage: richiede uno stack USB host e un MCU con OTG; molto più complesso per un vantaggio nullo a 10 Hz.

Acronimi:

- **SDIO** = Secure Digital Input/Output, interfaccia nativa a 4 bit delle schede SD
- **UBX** = protocollo binario proprietario u-blox
- **OTG** = On-The-Go, capacità USB di agire da host

Caso 4 - Gateway con modem cellulare LTE

Situazione. Un dispositivo raccoglie dati da un bus di campo e li invia in cloud tramite un modem LTE-M/NB-IoT (es. Quectel BG95, u-blox SARA).

- **MCU - modem: UART con controllo di flusso RTS/CTS.** I modem si comandano con stringhe AT e rispondono con testo: una console seriale a 115200 baud è l'interfaccia naturale. Il controllo di flusso hardware serve perché il modem può emettere raffiche di dati (URC, download) mentre l'MCU è occupato.
- **Lato bus di campo: RS-485/Modbus RTU**, come nel caso 1, con la USART in modalità DE automatico.

Perché non le alternative:

- USB verso il modem: molti modem lo offrono, ma richiede uno stack USB host con driver CDC sull'MCU: settimane di lavoro e MCU più grosso, per una banda che l'LTE-M non satura comunque (~375 kbit/s).
- SPI verso il modem: non offerto dai modem cellulari commerciali; il produttore decide l'interfaccia.
- Senza flow control: a 115200 baud un buffer da 256 byte si riempie in 22 ms; una ISR in ritardo (es. durante una scrittura in flash che blocca gli interrupt) causa overrun e perdita di risposte AT.

Acronimi:

- **LTE-M** = Long Term Evolution for Machines, profilo LTE a basso consumo
- **NB-IoT** = Narrowband Internet of Things
- **URC** = Unsolicited Result Code, messaggio spontaneo del modem
- **CDC** = Communications Device Class, classe USB per porte seriali virtuali

Caso 5 - Rete di nodi di controllo distribuiti su una macchina industriale

Situazione. Otto schede (azionamenti, I/O remoti, pannello operatore) distribuite su 30 m devono scambiarsi comandi e stati con latenza garantita di pochi millisecondi; ogni nodo deve poter parlare quando ha un evento, non solo quando interrogato.

- **CAN a 500 kbit/s.** Multi-master con arbitraggio hardware non distruttivo: il nodo con il messaggio più urgente (ID più basso) passa per primo senza collisioni. CRC, ACK, ritrasmissione automatica ed esclusione dei nodi guasti (bus off) sono nel silicio: il firmware riceve solo frame validi. Il livello fisico differenziale copre i 30 m con margine.
- **Sopra CAN: CANopen** o un protocollo proprietario a oggetti, per dare significato agli identificatori.

Perché non le alternative:

- RS-485 + Modbus: è master/slave. Il pannello dovrebbe interrogare a turno gli altri 7 nodi: un evento su un nodo aspetta il proprio turno (decine di ms a 115200 baud) e il rilevamento degli errori si ferma al CRC, senza ritrasmissione né isolamento del nodo guasto.
- Ethernet industriale (EtherNet/IP, PROFINET): più banda, ma costo per nodo (PHY, connettori, switch) e complessità software molto superiori; giustificato solo con dati voluminosi o integrazione IT.
- SPI/I2C: distanze di metri, nessun multi-master utilizzabile (I2C in teoria sì, in pratica no) e nessuna gestione errori.

Acronimi:

- **PROFINET** = Process Field Network, Ethernet industriale promosso da Siemens
- **IT** = Information Technology

Caso 6 - Firmware, aggiornamenti OTA e asset grafici in memoria esterna

Situazione. Un dispositivo con interfaccia grafica deve conservare 4 MB di immagini e font, più una seconda copia del firmware per l'aggiornamento remoto con rollback.

- **Flash NOR esterna su Quad SPI.** A 80 MHz in modalità quad si leggono ~40 MB/s: un'immagine full screen 320 x 240 x 16 bit (150 kB) arriva in ~4 ms. Con XIP il codice può girare direttamente dalla flash, e l'aggiornamento OTA scrive la partizione inattiva mentre quella attiva continua a funzionare.
- Su ESP32 questa architettura è nativa: flash e PSRAM esterne su SPI/QSPI, partizioni OTA gestite dal bootloader.

Perché non le alternative:

- EEPROM I2C: taglie massime di 256 kB-1 MB e ~40 kB/s: 4 MB di asset sarebbero 100 s di lettura. Va bene per configurazioni, non per dati.
- SPI a 1 bit: funziona (stesso chip, 1 linea dati), ma a 40 MHz servono 8 volte il tempo (1 bit per clock a metà frequenza): ~30 ms per immagine, visibili come sfarfallio negli aggiornamenti di schermo. Si usa quando i pin QSPI non ci sono.
- SD card: rimovibile, quindi inaffidabile per il firmware (può mancare o essere corrotta); più costosa in connettore e meccanica; le scritture casuali frequenti la usurano.

Acronimi:

- **OTA** = Over-The-Air, aggiornamento firmware da remoto

Caso 7 - Due microcontrollori sulla stessa scheda (MCU applicativo + coprocessore radio)

Situazione. Un STM32 gestisce misura e controllo in tempo reale; un ESP32 fa Wi-Fi e Bluetooth. Devono scambiarsi comandi e dati sulla stessa scheda.

- **Traffico modesto (comandi, telemetria a pochi kB/s): UART a 921600 baud.** Nessun rapporto master/slave, entrambi possono iniziare a trasmettere, nessuna linea di clock, 2 pin. Serve un framing applicativo (lunghezza + CRC, o COBS) e un protocollo di richiesta/risposta o eventi.

- **Traffico elevato (streaming audio, tunneling TCP/IP, es. ESP-Hosted): SPI a 20-40 MHz.** Decine di Mbit/s reali, DMA su entrambi i lati. Va aggiunto un GPIO 'data ready' dallo slave verso il master: in SPI lo slave non può prendere l'iniziativa, e il master non deve leggere quando lo slave non ha dati pronti.

Perché non le alternative:

- UART per lo streaming: 921600 baud sono ~90 kB/s netti, insufficienti per audio non compresso o per fare da ponte a una connessione Wi-Fi; oltre 1-2 Mbaud le tolleranze di clock dei due MCU diventano critiche.
- SPI per il traffico modesto: funziona, ma introduce asimmetria master/slave, un pin extra di handshake e la necessità di transazioni a lunghezza nota; più complesso per zero vantaggio a bassa banda.
- I2C: lo slave I2C in software su un MCU generico è delicato (clock stretching, timeout) e la banda è la più bassa; ha senso solo se l'MCU coprocessore si presenta già come 'periferica I2C' con un protocollo a registri.

Caso 8 - Pannello touch con display TFT e controller capacitivo

Situazione. Interfaccia utente con LCD 3,5 pollici a 320 x 480 pixel e touch capacitivo, pilotata da un MCU a 168 MHz.

- **Display: SPI a 40 MHz (o interfaccia parallela 8080 se il MCU la ha).** Un frame RGB565 è 307 kB; a 40 MHz servono ~61 ms, quindi si aggiornano solo le aree modificate (dirty rectangles) tramite comandi di finestra. Se serve fluidità video si passa a un controller LCD parallelo (LTDC su STM32F4/F7) con framebuffer in RAM.
- **Touch: I2C a 400 kHz.** Il controller (FT6206, GT911) produce poche decine di byte per evento e segnala il tocco con un pin INT: si legge solo quando serve. Condivide il bus con l'eventuale sensore di luce per la retroilluminazione.

Perché non le alternative:

- I2C per il display: 307 kB a 400 kHz sono ~7 s per frame: inutilizzabile. Anche i piccoli OLED I2C 128 x 64 (1 kB) richiedono ~25 ms per frame, accettabili solo per testo statico.
- SPI per il touch: alcuni controller lo offrono, ma costa un CS e non porta nulla: il collo di bottiglia è il dito, non il bus.
- UART per entrambi: nessun display o touch controller integrato lo usa; esistono 'display intelligenti' con UART (Nextion), ma spostano la grafica in un modulo esterno con logica propria, cambiando l'architettura.

Acronimi:

- **RGB565** = formato colore a 16 bit: 5 bit rosso, 6 verde, 5 blu
- **LTDC** = LCD-TFT Display Controller, periferica STM32 per display paralleli con framebuffer

Schema di ragionamento da riusare

Di fronte a un nuovo componente o collegamento, le domande in ordine: (1) **cosa offre il componente?** Spesso decide lui. (2) **Quanti byte al secondo**, davvero? Calcolarlo: se stanno in I2C a 400 kHz con margine 10x, I2C. (3) **Quanti pin** ho e quanti dispositivi devo collegare? (4) **Che distanza:** sulla scheda, nel cabinet, tra edifici? Sopra i 50 cm serve un transceiver, quindi UART/RS-485 o CAN. (5) **Chi deve poter iniziare** a parlare? Se più nodi, CAN (o UART tra due). (6) **Cosa succede se un byte si perde?** Se conta, servono CRC e ritrasmissione: nativi in CAN, da scrivere altrove. (7) **Consumo:** tempo attivo del bus e pull-up. La risposta 'giusta' al colloquio è la sequenza di queste domande applicata al caso proposto, non un protocollo preferito a priori.

6. Cenni a RS-485, Modbus RTU e CAN

Questi tre argomenti sono naturali estensioni della domanda su UART e sono molto rilevanti in ambito industriale ed energy metering.

6.1 RS-485 (TIA/EIA-485-A)

- **Standard elettrico**, non protocollo: definisce un driver/receiver **differenziale** su doppino (A/B, o D+/D-). Il bit è codificato come differenza di tensione (tipicamente $\geq \pm 200$ mV al ricevitore, $\geq 1,5$ V al driver): il rumore di modo comune (fino a ± 12 V) si cancella.
- **Multidrop**: fino a 32 'unit load' sullo stesso bus (256 con transceiver a 1/8 UL). Topologia a bus lineare, **terminazione 120 Ω** alle due estremità (uguale all'impedenza caratteristica del cavo) per evitare riflessioni; resistenze di **bias** (fail-safe) per definire lo stato idle quando nessun driver è attivo.
- **Distanza vs velocità**: fino a ~ 1200 m a 100 kbit/s, ~ 10 Mbit/s su pochi metri (regola empirica: velocità x lunghezza $\leq 10^8$ bit/s x m).

Acronimi:

- **TIA/EIA** = Telecommunications Industry Association / Electronic Industries Alliance

Calcolo: regola empirica velocità x lunghezza su RS-485

Regola pratica (non da specifica, ma usata nelle application note dei transceiver): baud x lunghezza $\leq 10^8$ bit/s x m

A 1200 m: $\text{baud}_{\max} = 10^8 / 1200 = \sim 83 \text{ kbit/s}$ -> si usa 9600 o 19200 con ampio margine

A 100 m: $10^8 / 100 = 1 \text{ Mbit/s}$

A 10 m: $10^8 / 10 = 10 \text{ Mbit/s}$ (limite del transceiver, non del cavo)

Il limite fisico sottostante: il tempo di propagazione sul cavo (~ 5 ns/m su doppino, $v = 0,66 c$) deve restare piccolo rispetto al tempo di bit. A 1200 m: 6 μs di propagazione contro 104 μs di bit a 9600 baud (6 %), contro 8,7 μs a 115200 baud (69 %: fronti riflessi e attenuazione compromettono il campionamento).

Tempo di bit: $T_{\text{bit}} = 1 / \text{baud}$ -> 9600 baud = 104,2 μs ; 19200 = 52,1 μs ; 115200 = 8,68 μs

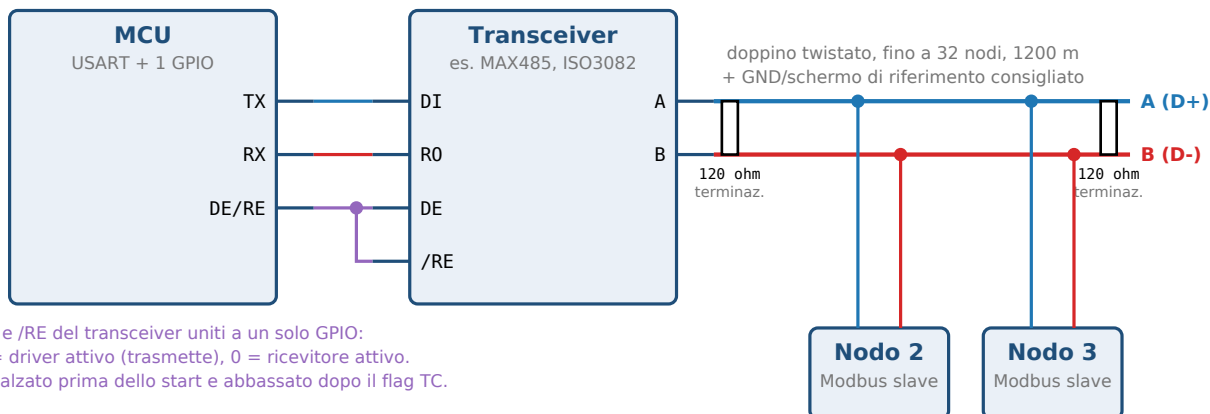
- **Half duplex** a 2 fili (il più comune): un solo nodo trasmette alla volta, il firmware pilota il pin **DE** (Driver Enable) del transceiver prima di trasmettere e lo rilascia subito dopo l'ultimo stop bit. Le USART STM32 hanno il DE automatico con tempi di assertion/deassertion configurabili; su ESP32 esiste UART_MODE_RS485_HALF_DUPLEX. Esiste anche il 4 fili full duplex (spesso chiamato RS-422 se punto-punto).
- **Isolamento galvanico** (transceiver isolati, es. ADM2587E, ISO3082) e protezione TVS sono standard nei prodotti industriali per superare le prove di immunità (EN 61000-4-x).

Acronimi:

- **TVS** = Transient Voltage Suppressor, diodo di protezione dai transitori
- **EN** = European Norm, norma europea

Con DE automatico della USART (pin RTS): sempre 3 pin fisici, ma 0 GPIO gestiti a firmware.

Pin MCU: 3 (TX, RX, DE)



DE e /RE del transceiver uniti a un solo GPIO:

1 = driver attivo (trasmette), 0 = ricevitore attivo.

Va alzato prima dello start e abbassato dopo il flag TC.

RS-485 half duplex a 2 fili: la UART dell'MCU esce su un doppino differenziale multidrop attraverso un transceiver.

Figura 7 - Nodo RS-485: USART + GPIO di direzione verso un transceiver; bus a 2 fili terminato ai capi.

6.2 Modbus RTU

Protocollo applicativo (Modicon, 1979, oggi Modbus Organization) **master/slave** - oggi client/server - trasportato su UART/RS-485. Semplice, aperto, onnipresente in PLC, inverter, contatori di energia, HVAC. La variante **Modbus TCP** incapsula la stessa PDU in TCP/IP; **Modbus ASCII** è la versione testuale, ormai rara.

Acronimi:

- **HVAC** = Heating, Ventilation and Air Conditioning
- **PDU** = Protocol Data Unit
- **TCP/IP** = Transmission Control Protocol / Internet Protocol
- **ASCII** = American Standard Code for Information Interchange

Elemento	Dettaglio
Frame RTU	Address (1 byte) + Function code (1 byte) + Data (0-252 byte) + CRC-16 (2 byte, low byte first). Max 256 byte totali.
Indirizzi	1-247 per gli slave; 0 = broadcast (solo scritture, nessuna risposta); 248-255 riservati.
Delimitazione	Basata sul silenzi o: inizio/fine frame = pausa $\geq$ 3,5 caratteri; una pausa $>$ 1,5 caratteri all'interno di un frame lo invalida. Sopra 19200 baud la specifica fissa 1,75 ms e 750 μ s. Si implementa con timer o con l'IDLE detection della USART.
Formato seriale	Default di specifica: 19200 baud, 8E1 (11 bit per carattere). Molto diffuso anche 9600 8N1/8N2. Con parity None la specifica richiede 2 stop bit per mantenere 11 bit.
CRC-16	Polinomio 0x8005 riflesso (0xA001), valore iniziale 0xFFFF, calcolato su address+function+data. Tabella o bit-a-bit.
Modello dati	4 tabelle: Coils (1 bit R/W), Discrete Inputs (1 bit R), Input Registers (16 bit R), Holding Registers (16 bit R/W). Indirizzi 0-65535 per tabella, con la notazione 'offset+1' (40001 = holding register 0) che genera confusione.
Function code principali	0x01 Read Coils, 0x02 Read Discrete Inputs, 0x03 Read Holding Registers , 0x04 Read Input Registers , 0x05 Write Single Coil, 0x06 Write Single Register , 0x0F Write Multiple Coils, 0x10 Write Multiple Registers , 0x17 Read/Write Multiple.
Eccezioni	Risposta con function code 0x80 e un codice: 01 Illegal Function, 02 Illegal Data Address, 03 Illegal Data Value, 04 Slave Device Failure, 06 Slave Device Busy.
Valori a 32 bit	Non standardizzati: due registri consecutivi, con ordine delle word (big/little endian, 'word swap') dipendente dal costruttore. Va documentato nella register map.
Timing lato slave	Rispondere entro il timeout del master (tipicamente 100 ms - 1 s), dopo la pausa di 3,5 caratteri. La gestione del DE deve essere precisa per non tronc

Calcolo: tempi di silenzio Modbus RTU ($t_{3,5}$ e $t_{1,5}$)

Un carattere Modbus RTU è sempre di **11 bit**: 1 start + 8 dati + 1 parità + 1 stop (oppure 8N2)

Tempo di un carattere: $T_{\text{char}} = 11 / \text{baud}$

A 9600 baud: $T_{\text{char}} = 11 / 9600 = 1,146 \text{ ms} \rightarrow t_{3,5} = 3,5 \times 1,146 = \mathbf{4,01 \text{ ms}}$; $t_{1,5} = 1,5 \times 1,146 = \mathbf{1,72 \text{ ms}}$

A 19200 baud: $T_{\text{char}} = 0,573 \text{ ms} \rightarrow t_{3,5} = \mathbf{2,00 \text{ ms}}$; $t_{1,5} = \mathbf{0,86 \text{ ms}}$

Sopra 19200 baud la specifica **fissa** i valori a $t_{3,5} = 1,75 \text{ ms}$ e $t_{1,5} = 750 \mu\text{s}$ (a 115200 il calcolo darebbe 334 μs e 143 μs , troppo stretti per gli MCU del 1979)

Durata di una richiesta 0x03 (8 byte: addr, fc, start hi/lo, qty hi/lo, crc lo/hi) a 9600: $8 \times 1,146 = \mathbf{9,2 \text{ ms}}$; risposta con 10 registri (3 + 20 + 2 = 25 byte): **28,6 ms**

Ciclo completo richiesta + $t_{3,5}$ + elaborazione slave (es. 5 ms) + risposta + $t_{3,5} = 9,2 + 4 + 5 + 28,6 + 4 = \mathbf{\sim 51 \text{ ms}}$ -> su un bus a 9600 con 10 slave da interrogare, il periodo di polling minimo è circa 0,5 s. È il motivo per cui si sale a 19200/38400 quando i nodi sono molti.

Calcolo: CRC-16 Modbus

Polinomio $x^{16} + x^{15} + x^2 + 1 = 0x8005$; in forma riflessa (LSB first, come i bit UART) = **0xA001**; valore iniziale **0xFFFF**; nessuno XOR finale

Per ogni byte: $\text{crc} \oplus \text{byte}$; poi 8 volte: se $(\text{crc} \& 1)$ allora $\text{crc} = (\text{crc} \gg 1) \oplus 0xA001$, altrimenti $\text{crc} = \text{crc} \gg 1$

Il risultato si trasmette **low byte prima** (unico campo little-endian del frame; i registri dati sono big-endian)

Verifica rapida: il frame 01 03 00 00 00 0A ha CRC = **C5 CD** (traspresso: 01 03 00 00 00 0A C5 CD)

Il CRC-16 rileva tutti gli errori su 1 e 2 bit, tutti quelli su un numero impari di bit e tutti i burst fino a 16 bit; non rileva 1 errore su 65536 pattern casuali più lunghi.

Acronimi:

- **XOR** = OR esclusivo

6.3 CAN (Controller Area Network)

Sviluppato da Bosch (1986) per l'automotive, oggi ISO 11898. È un bus **multi-master a messaggi**, non a indirizzi di nodo: ogni frame porta un **identificatore** che ne definisce contenuto e priorità, e tutti i nodi lo ricevono filtrando ciò che interessa.

- **Livello fisico** (ISO 11898-2, high-speed): doppino differenziale CAN_H/CAN_L, terminazione 120 Ω ai capi, fino a 1 Mbit/s (40 m) o 125 kbit/s (500 m). Livello **dominante** (0, CAN_H - CAN_L circa 2 V) e **recessivo** (1, differenza circa 0 V): il dominante vince sempre, come lo 0 in I2C.
- **Arbitraggio**: i nodi che iniziano a trasmettere contemporaneamente confrontano bit a bit l'ID; chi trasmette recessivo e legge dominante perde e si ritira. **ID più basso = priorità più alta**. Non distruttivo, deterministico: il messaggio prioritario non subisce ritardo.
- **Frame**: SOF, ID a 11 bit (standard, CAN 2.0A) o 29 bit (extended, 2.0B), RTR, DLC (0-8 byte in CAN classico, fino a 64 in **CAN FD** dove la fase dati può salire a 2-8 Mbit/s), CRC-15, slot **ACK** (ogni ricevitore corretto scrive dominante), EOF. **Bit stuffing**: dopo 5 bit uguali il trasmettitore inserisce un bit opposto per mantenere la sincronizzazione (codifica NRZ senza clock).
- **Gestione errori in hardware**: 5 tipi (bit, stuff, CRC, form, ACK). Un nodo che rileva un errore trasmette un **error frame** che invalida il messaggio per tutti; il trasmettitore ritrasmette automaticamente. Contatori TEC/REC portano il nodo negli stati **error active** -> **error passive** -> **bus off** (il nodo guasto si autoesclude: fault confinement).

Acronimi:

- **SOF** = Start Of Frame
- **RTR** = Remote Transmission Request
- **DLC** = Data Length Code
- **FD** = Flexible Data-rate
- **EOF** = End Of Frame
- **NRZ** = Non-Return-to-Zero, codifica di linea
- **TEC** = Transmit Error Counter
- **REC** = Receive Error Counter

Calcolo: bit timing CAN (esempio 500 kbit/s con clock periferica 42 MHz)

Il bit è diviso in **time quanta** (tq): $T_{\text{bit}} = tq \times (1 + BS1 + BS2)$, dove 1 è il Sync_Seg, BS1 = Prop_Seg + Phase_Seg1, BS2 = Phase_Seg2

tq = prescaler / f_{CAN} . Obiettivo: $T_{\text{bit}} = 1 / 500\,000 = 2\,\mu\text{s}$

Scelta: prescaler = 6 -> tq = 6 / 42 000 000 = 142,86 ns -> tq per bit = 2 μs / 142,86 ns = **14**

Sample point tipico 75-87,5 %: con BS1 = 11, BS2 = 2 -> (1 + 11) / 14 = **85,7 %**. Verifica: 142,86 ns x 14 = 2,000 μs -> baud effettivo 500 000, errore 0 %

Il sample point deve essere **uguale su tutti i nodi** del bus (a parità di baud rate, nodi con sample point diverso possono generare errori sporadici).

Lunghezza massima del bus: entro un bit il segnale deve fare andata e ritorno (arbitraggio e ACK richiedono che tutti vedano lo stesso bit). Con ~5 ns/m + ritardi dei transceiver (~2 x 100 ns): a 1 Mbit/s (1 μs) restano ~600 ns per 2 x L -> L <= ~**40 m**; a 125 kbit/s (8 μs) -> ~**500 m**.

Acronimi:

- **tq** = time quantum, quanto temporale del bit CAN
- **BS1** = Bit Segment 1
- **BS2** = Bit Segment 2

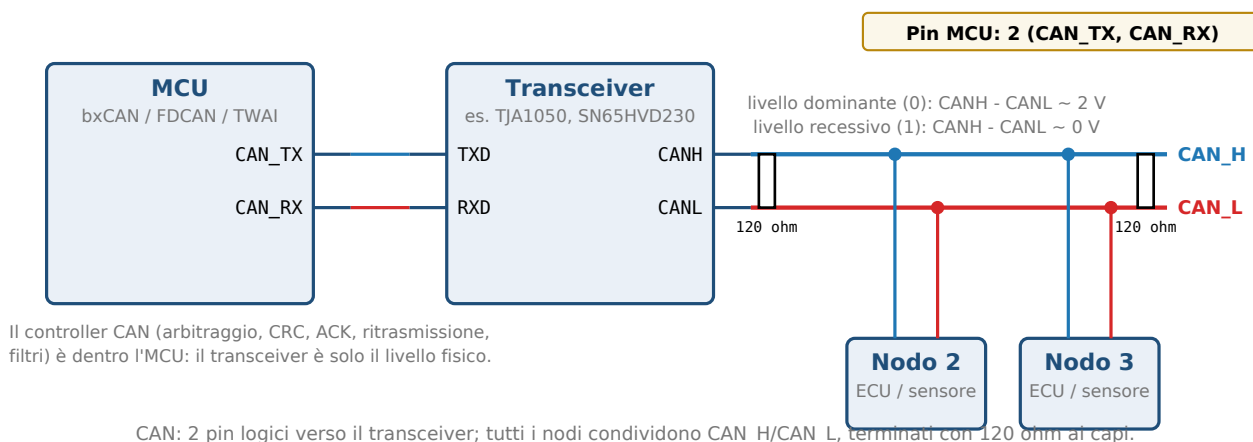


Figura 8 - Nodo CAN: controller integrato nell'MCU (2 pin) + transceiver esterno verso CAN_H/CAN_L.

- **Firmware:** la periferica (bxCAN, FDCAN su STM32; TWAI su ESP32) gestisce tutto il livello data link; il firmware configura bit timing (prescaler, segmenti, sample point tipicamente 75-87,5 %) e filtri di accettazione, poi legge/scrive mailbox. Serve sempre un **transceiver** esterno (TJA1050, MCP2551, SN65HVD230). Protocolli applicativi: CANopen, J1939, ISO-TP/UDS, DeviceNet (che è CAN + CIP, parente di EtherNet/IP).

Acronimi:

- **bxCAN** = Basic Extended CAN, periferica CAN STM32
- **FD CAN** = Flexible Data-rate CAN, periferica CAN FD STM32
- **TWAI** = Two-Wire Automotive Interface, nome Espressif del CAN
- **CANopen** = protocollo applicativo CAN in Automation
- **J1939** = SAE J1939, protocollo CAN per veicoli pesanti
- **ISO-TP** = ISO Transport Protocol
- **UDS** = Unified Diagnostic Services
- **CIP** = Common Industrial Protocol

7. Domande tipiche da colloquio (con risposte)

Le risposte sono volutamente sintetiche, come si darebbero a voce: partire dal concetto chiave e aggiungere dettagli se l'intervistatore approfondisce.

Generali

Q1. Che differenza c'è tra comunicazione sincrona e asincrona? Quali vantaggi ha ciascuna?

Nella sincrona c'è una linea di clock condivisa (SPI, I2C): il ricevitore campiona sui fronti, non serve conoscere la velocità a priori e si possono raggiungere frequenze elevate; costa un pin in più. Nell'asincrona (UART) non c'è clock: i due nodi concordano un baud rate e si risincronizzano sullo start bit di ogni frame; risparmia un filo ma richiede oscillatori precisi (errore complessivo sotto il 2-3 %) e ha overhead di start/stop.

Q2. Perché I2C usa uscite open-drain mentre SPI e UART usano push-pull?

Perché su I2C più dispositivi condividono le stesse linee e devono poterle pilotare senza conflitti: con l'open-drain nessuno può forzare un 1 contro uno 0 altrui, quindi si evitano corti, si abilita l'arbitraggio multi-master e il clock stretching. Il prezzo è il fronte di salita lento (RC del pull-up), che limita la velocità. SPI e UART hanno un solo driver per linea in ogni istante (MISO va in tri-state quando lo slave non è selezionato) e possono permettersi il push-pull, veloce e a basso consumo.

Q3. Come si sceglie tra polling, interrupt e DMA per una periferica seriale?

Polling per test o traffici minimi; interrupt quando i byte sono pochi e sporadici o servono reazioni immediate (es. parser carattere per carattere); DMA quando il traffico è continuo o a blocchi (frame Modbus, buffer display SPI, log su UART) per liberare la CPU e azzerare il rischio di overrun. Il pattern più robusto per UART è DMA circolare + interrupt IDLE (o timeout carattere) per delimitare i pacchetti a lunghezza variabile.

Q4. Un dispositivo seriale non risponde: come procedi?

In ordine: alimentazione e GND comune; connessioni (TX/RX incrociati, CS giusto, pull-up presenti); configurazione (baud rate, modo SPI, indirizzo I2C shiftato o no); poi oscilloscopio o analizzatore logico per vedere se il master genera davvero il traffico e se lo slave risponde (ACK su I2C, dati su MISO). Infine il datasheet: sequenze di power-up, tempi minimi tra CS e clock, bit necessari per il multi-byte read.

UART

Q5. Cosa succede se i due nodi UART hanno baud rate leggermente diversi? Quanto errore è tollerabile?

Il ricevitore campiona ogni bit al centro partendo dallo start bit; l'errore si accumula lungo il frame e sull'ultimo bit (stop) il campionamento può cadere fuori. Con 10 bit per frame e campionamento centrale, l'errore accumulato deve restare sotto mezzo bit: circa il 5 % teorico, ma considerando oversampling, jitter e tolleranza di entrambi i lati si sta in pratica sotto il 2-3 % complessivi. Un baud rate sbagliato produce tipicamente framing error e caratteri spazzatura.

Q6. Che differenza c'è tra UART, RS-232 e RS-485?

UART è la periferica/formato del frame a livello logico. RS-232 e RS-485 sono standard elettrici per trasportare quei frame su cavo: RS-232 single-ended con tensioni bipolari (+/-3 a +/-15 V, logica invertita), punto-punto, corto raggio; RS-485 differenziale, multidrop fino a 32 nodi, 1200 m, robusto al rumore, tipicamente half duplex con controllo del driver enable.

Q7. Come delimiti i messaggi su una UART, che è un flusso di byte senza struttura?

Quattro tecniche: terminatore (es. '\r\n' o 0x00 con COBS/SLIP per dati binari), campo lunghezza in header con eventuale sync byte e CRC, timeout/gap tra byte (come Modbus RTU con i 3,5 caratteri, implementabile con l'IDLE line detection), o frame a lunghezza fissa. In ogni caso serve una macchina a stati che si risincronizzi dopo un byte corrotto.

Q8. Cos'è l'overrun error e come lo si evita?

Arriva un nuovo byte mentre il precedente non è ancora stato letto dal registro dati: il dato è perso e su STM32 il flag ORE blocca ulteriori ricezioni finché non viene cancellato. Si evita con DMA, FIFO hardware, ISR corte, e con una gestione degli errori che cancelli il flag (HAL_UART_ErrorCallback) e riarmi la ricezione.

SPI

Q9. Spiega CPOL e CPHA. Cosa succede se sono sbagliati?

CPOL definisce il livello di riposo del clock, CPHA su quale fronte (primo o secondo) si campiona il dato. I quattro modi 0-3 ne sono le combinazioni; il modo 0 (idle basso, campionamento sul fronte di salita) è il più comune. Con il modo sbagliato i dati sono letti sul fronte in cui stanno cambiando: si ottengono valori shiftati di un bit, 0xFF/0x00 o spazzatura. Alcuni dispositivi accettano sia il modo 0 sia il 3 perché campionano sullo stesso fronte di salita.

Q10. Perché per leggere da SPI devi trasmettere byte dummy?

Perché SPI è uno scambio simmetrico tra due shift register: il clock lo genera solo il master e ogni impulso sposta un bit in entrambe le direzioni. Se il master vuole ricevere deve generare clock, e generare clock significa trasmettere quello che ha nel registro. Il contenuto dei dummy è indifferente per la maggior parte dei dispositivi, ma alcuni lo interpretano come comando: va scelto secondo datasheet (0xFF per le SD card).

Q11. Come gestisci più slave SPI e quali problemi possono sorgere?

Un CS per slave, con SCK/MOSI/MISO condivisi. Problemi: un solo slave alla volta selezionato (mai due CS bassi insieme), MISO che deve essere tri-state quando non selezionato, modi di clock diversi tra dispositivi (riconfigurare la periferica prima di ogni transazione o usare bus separati), velocità massima diversa, capacità del bus che cresce con i dispositivi. In alternativa daisy chain se i dispositivi lo supportano.

Q12. Cos'è il Quad SPI e a cosa serve l'XIP?

Quad SPI usa 4 linee dati bidirezionali invece di MOSI/MISO, quadruplicando la banda verso memorie flash esterne. XIP (execute in place) permette alla CPU di eseguire codice mappato direttamente sulla flash esterna tramite la periferica QSPI con cache, senza copiarlo in RAM: è il modo in cui funzionano gli ESP32, la cui flash è tutta esterna al die.

I2C

Q13. Descrivi una transazione I2C di lettura di un registro.

START, indirizzo a 7 bit + W, ACK dello slave, byte con l'indirizzo del registro, ACK, repeated START, indirizzo + R, ACK, lo slave trasmette i byte con auto-incremento e il master fa ACK su ciascuno tranne l'ultimo, sul quale manda NACK per dire 'ho finito', poi STOP. Il repeated START evita di rilasciare il bus tra la scrittura del puntatore e la lettura: in multi-master garantisce che nessun altro modifichi il puntatore nel frattempo.

Q14. Come dimensioni le resistenze di pull-up?

Minimo dalla corrente massima di sink dell'uscita (3 mA, VOL 0,4 V): a 3,3 V circa 1 kohm. Massimo dal tempo di salita ammesso per il modo (1000 ns Sm, 300 ns Fm, 120 ns Fm+) e dalla capacità del bus: $R_{max} = t_r / (0,8473 \times C_b)$. Con 100 pF a 400 kHz si arriva a circa 3,5 kohm; in pratica 4,7 kohm per 100 kHz, 2,2 kohm per 400 kHz, poi si verifica il fronte con l'oscilloscopio.

Q15. Cos'è il clock stretching e perché può dare problemi?

Lo slave tiene SCL bassa dopo l'ACK per segnalare che non è pronto a fornire il dato; il master deve aspettare che SCL torni alta prima di continuare. I controller hardware lo gestiscono, ma implementazioni software o alcuni master (storicamente il Raspberry Pi) no, causando letture corrotte. Alcuni slave non lo generano e quindi vanno interrogati con tempi adeguati.

Q16. Il bus I2C è bloccato con SDA bassa: cosa è successo e come lo risolvi?

Tipicamente il master si è resettato a metà di una lettura: lo slave sta trasmettendo uno 0 e aspetta un clock che non arriva; il master non può generare START perché SDA è già bassa. Recovery: configurare SCL come GPIO e generare fino a 9 impulsi finché SDA si libera, poi generare uno STOP manuale e reinizializzare la periferica. Va fatto all'avvio e dopo un timeout. Prevenzione: watchdog, timeout di 35 ms come in SMBus, reset dei sensori via pin dedicato.

Q17. Perché la HAL STM32 vuole l'indirizzo I2C a 8 bit?

Perché il primo byte trasmesso sul bus è indirizzo a 7 bit seguito dal bit R/W: la HAL prende direttamente quel byte e imposta lei il bit 0. Quindi 0x68 va passato come 0xD0 (0x68 << 1). ESP-IDF, Linux e la maggior parte delle librerie Arduino usano invece il 7 bit non shiftato. Se un dispositivo 'non risponde' ma fa ACK a un indirizzo doppio o dimezzato, è questo.

Q18. Due sensori uguali con lo stesso indirizzo: cosa fai?

Uso i pin di selezione indirizzo se il chip li ha (spesso 1-2 bit), altrimenti un multiplexer I2C (TCA9548A) o un secondo bus del microcontrollore; in extremis un GPIO che alimenta/abilita un sensore alla volta. Da evitare soluzioni a bit-bang duplicato.

RS-485 / Modbus / CAN

Q19. Come gestisci il Driver Enable su RS-485 e quali errori sono tipici?

Alzo DE subito prima del primo start bit e lo abbasso solo dopo che l'ultimo stop bit è uscito fisicamente dal pin (flag TC 'transmission complete', non TXE che indica solo registro vuoto). Abbassarlo troppo presto tronca l'ultimo byte; troppo tardi si sovrappone alla risposta dello slave. Le USART moderne hanno il DE automatico. Altri errori: manca la terminazione, mancano i bias e il ricevitore vede rumore in idle, A/B invertiti, mancanza di riferimento di massa comune.

Q20. Perché in CAN l'ID basso ha priorità?

Perché durante l'arbitraggio il bit dominante è lo 0: un nodo che trasmette 1 (recessivo) e rilegge 0 capisce che un altro sta trasmettendo un ID con uno 0 in quella posizione, quindi più basso, e si ritira. Chi ha più zeri nei bit alti vince. L'arbitraggio è non distruttivo: il vincitore prosegue senza ritardo, il perdente ritrasmette dopo.

Q21. Come si delimitano i frame in Modbus RTU e come lo implementi?

Con il silenzio sul bus: almeno 3,5 tempi carattere prima e dopo il frame (1,75 ms sopra 19200 baud). Si implementa con un timer riarmato a ogni byte ricevuto o, più semplicemente, con l'interrupt IDLE della UART + DMA: allo scadere si valida lunghezza minima, indirizzo e CRC-16 e si passa il frame al livello applicativo. La risposta deve rispettare lo stesso gap.

8. Riferimenti

Fonti primarie consigliate per approfondire e verificare i valori numerici citati. **Nota:** questo documento è stato redatto in una sessione senza accesso alla rete, quindi i riferimenti provengono dalla conoscenza dell'autore e non sono stati riverificati online al momento della stesura; i valori di specifica (velocità I2C, capacità di bus, timing Modbus, ecc.) corrispondono alle revisioni delle specifiche note e vanno confermati sul documento originale prima di citarli in contesti formali.

- **I2C:** NXP, *UM10204 - I2C-bus specification and user manual*, Rev. 7 (2021).
<https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- **I2C pull-up:** Texas Instruments, *SLVA689 - I2C Bus Pull-Up Resistor Calculation*.
<https://www.ti.com/lit/an/slva689/slva689.pdf>
- **SPI:** nessuna specifica ufficiale; riferimento storico Motorola *SPI Block Guide V03.06*. Per le implementazioni: reference manual del MCU (es. STMicroelectronics RM0090 per STM32F4, capitolo SPI).
- **UART/USART:** reference manual STM32 (es. RM0090 cap. USART, con tabelle di tolleranza del baud rate); Espressif *ESP32 Technical Reference Manual*, cap. UART.
- **Driver ESP-IDF:**
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/>
(uart.html, spi_master.html, i2c.html). L'API I2C è cambiata in ESP-IDF 5.2: verificare la versione in uso.
- **HAL STM32:** STMicroelectronics *UM1725 - Description of STM32F4 HAL and low-layer drivers* (o equivalente per la famiglia usata).
- **RS-485:** TIA/EIA-485-A; Texas Instruments *SLLA070 - RS-422 and RS-485 Standards Overview and System Configurations*; Maxim/ADI *AN723 - Guidelines for Proper Wiring of an RS-485 Network*.
- **Modbus:** Modbus Organization, *MODBUS Application Protocol Specification V1.1b3* e *MODBUS over Serial Line Specification and Implementation Guide V1.02*. <https://modbus.org/specs.php>
- **CAN:** ISO 11898-1:2015 (data link), ISO 11898-2:2016 (high-speed physical layer); Bosch *CAN Specification 2.0*; Texas Instruments *SLOA101 - Introduction to the Controller Area Network*.
- **SMBus:** System Management Interface Forum, *SMBus Specification 3.x*.
<https://smbus.org/specs/>

9. Glossario degli acronimi

Tutti gli acronimi usati nel documento, in ordine alfabetico. Ogni acronimo è comunque esplicitato alla sua prima occorrenza nel testo, inline o nella legenda che segue l'elemento in cui compare.

Acronimo	Significato	Acronimo	Significato
/RE	Receiver Enable (attivo basso)	NAND	flash a celle NAND
ACK	Acknowledge	NE	Noise Error
ADC	Analog-to-Digital Converter	NMEA	National Marine Electronics Association
API	Application Programming Interface	NOR	flash a celle NOR
ASCII	American Standard Code for Information Interchange	NRZ	Non-Return-to-Zero
AT	ATtention (comandi Hayes)	NSS	Negative Slave Select (SPI STM32)
BRR	Baud Rate Register (STM32)	ODR	Output Data Rate
BS1	Bit Segment 1 (CAN)	OLED	Organic Light-Emitting Diode
BS2	Bit Segment 2 (CAN)	ORE	Overrun Error
bxCAN	Basic Extended CAN (STM32)	OVER8	bit oversampling 8 (USART STM32)
CAN	Controller Area Network	PC	Personal Computer
CANopen	protocollo applicativo CAN in Automation	PCB	Printed Circuit Board
CE	Chip Enable	PCLK	Peripheral Clock
CIP	Common Industrial Protocol	PDU	Protocol Data Unit
CIPO	Controller In Peripheral Out	PE	Parity Error
CK	Clock (uscita USART sincrona)	PEC	Packet Error Code
COBS	Consistent Overhead Byte Stuffing	PHY	Physical layer transceiver
COPI	Controller Out Peripheral In	PLC	Programmable Logic Controller
CPHA	Clock Phase	PMBus	Power Management Bus
CPOL	Clock Polarity	PMIC	Power Management Integrated Circuit
CPU	Central Processing Unit	ppm	parti per milione
CRC	Cyclic Redundancy Check	PSRAM	Pseudo-Static RAM
CS	Chip Select	PWM	Pulse-Width Modulation
CS_n / nCS	Chip Select negato (attivo basso)	QSPI	Quad SPI
CTS	Clear To Send	R/W	Read/Write
CubeMX	STM32CubeMX (tool di configurazione ST)	RAM	Random Access Memory
DAC	Digital-to-Analog Converter	RC	Resistenza-Condensatore
DDC	Display Data Channel	REC	Receive Error Counter
DE	Driver Enable	RFID	Radio-Frequency IDentification
DLC	Data Length Code	RS-232	Recommended Standard 232 (EIA/TIA)
DMA	Direct Memory Access	RS-422	Recommended Standard 422
ECU	Electronic Control Unit	RS-485	Recommended Standard 485
EDID	Extended Display Identification Data	RTC	Real-Time Clock
EEPROM	Electrically Erasable Programmable Read-Only Memory	RTR	Remote Transmission Request
EN	European Norm	RTS	Request To Send
EOF	End Of Frame	RTU	Remote Terminal Unit (Modbus binario)
ESP-IDF	Espressif IoT Development Framework	RX	Receive

Acronimo	Significato	Acronimo	Significato
FD	Flexible Data-rate (CAN FD)	S / Sr / P	START / repeated START / STOP (I2C)
FDCAN	Flexible Data-rate CAN (STM32)	SCK	Serial Clock (SPI)
FE	Framing Error	SCL	Serial Clock (I2C)
FIFO	First In First Out	SD	Secure Digital (card)
fps	frame per secondo	SDA	Serial Data (I2C)
FRAM	Ferroelectric RAM	SDI	Serial Data In
FTDI	Future Technology Devices International	SDO	Serial Data Out
GND	Ground	SLIP	Serial Line Internet Protocol
GPIO	General Purpose Input/Output	Sm/Fm/Fm+ /Hs/UFm	Standard-, Fast-, Fast-mode Plus, High-speed, Ultra Fast-mode (I2C)
GPS	Global Positioning System	SMBus	System Management Bus
GSM	Global System for Mobile Communications	SOF	Start Of Frame
HAL	Hardware Abstraction Layer (ST)	SPI	Serial Peripheral Interface
HVAC	Heating, Ventilation and Air Conditioning	SS	Slave Select
I/O	Input/Output	SSI	Synchronous Serial Interface (TI)
I2C	Inter-Integrated Circuit	STE	Slave Transmit Enable
ID	Identifier	TC	Transmission Complete (flag USART)
IMU	Inertial Measurement Unit	TCP/IP	Transmission Control Protocol / Internet Protocol
INT	Interrupt	TEC	Transmit Error Counter
IOL	corrente di uscita a livello basso	TFT	Thin-Film Transistor
IoT	Internet of Things	TIA/EIA	Telecommunications Industry Association / Electronic Industries Alliance
IrDA	Infrared Data Association	tq	time quantum (CAN)
ISO	International Organization for Standardization	TTL	Transistor-Transistor Logic
ISO-TP	ISO Transport Protocol	TVS	Transient Voltage Suppressor
ISR	Interrupt Service Routine	TWAI	Two-Wire Automotive Interface (ESP32)
J1939	SAE J1939 (CAN per veicoli pesanti)	TX	Transmit
LCD	Liquid Crystal Display	TXE	Transmit data register Empty (flag USART)
LED	Light-Emitting Diode	UART	Universal Asynchronous Receiver/Transmitter
LIN	Local Interconnect Network	UDS	Unified Diagnostic Services
LoRa	Long Range (modulazione radio)	UL	Unit Load (RS-485)
LSB	Least Significant Bit	USART	Universal Synchronous/Asynchronous Receiver/Transmitter
LTE	Long Term Evolution	USB	Universal Serial Bus
MCU	Microcontroller Unit	VDD	tensione di alimentazione positiva
MDIO	Management Data Input/Output	VOL	Voltage Output Low
MISO	Master In Slave Out	WHO_AM_I	registro di identificazione di un sensore
MOSI	Master Out Slave In	XIP	eXecute In Place
MSB	Most Significant Bit	XON/XOFF	Transmit On / Transmit Off
NACK	Not Acknowledge	XOR	OR esclusivo

Consiglio finale per il colloquio

Per ogni protocollo tenere pronti: (1) le linee e il loro ruolo, (2) come si sincronizzano i bit, (3) come si gestiscono più dispositivi, (4) come si rilevano gli errori, (5) un caso d'uso concreto del proprio lavoro (es. front-end di misura via SPI, EEPROM di calibrazione via I2C, Modbus RTU su RS-485 verso un PLC) e (6) un bug reale risolto (indirizzo I2C shiftato, modo SPI sbagliato, DE RS-485 rilasciato troppo presto). Gli esempi vissuti valgono più delle definizioni.